Agent Slicing: Localizing Blind Spots in Declared-Contract Models of Multi-Agent AI Teams

ANONYMOUS AUTHORS

Software teams increasingly build systems in which several AI agents hand work to one another. Such a team can be unsafe even when every agent meets its own contract, because the defect lives in a relationship among agents: a join that releases on the first approval, two reviewers that share one model, a test gate whose input was written by a fallible agent. We study this problem on *declared-contract models* of agent teams: directed acyclic graphs of agents and deterministic gates with requirements, guarantees, fork, switch, join policies (all, any, quorum) and model failures. We introduce *agent slicing*. A slice is a convex, possibly disconnected, set of agents analyzed exactly while the rest of the team is taken at face value. A slice reports a defect only when its witness *lifts* to a run of the whole team, so reported findings are never false alarms with respect to the model. The size of the smallest slice that confirms a defect is its *depth*; the smallest confirming slices localize it, and depth at least two means that an agent-by-agent check does not report it. We prove that confirmed findings are sound and monotone in the slice, when slice abstraction is compositional (partitions with an acyclic quotient and a global failure set), and that a change invalidates only a computable set of slices, and we give the complexity of each question. On 1,680 teams with defects induced by random engineering changes, 36.1% of the defect symptoms have depth at least two; for 210 of the 1,345 defective teams an agent-by-agent check reports nothing at all. A linear global must-analysis detects 100.0% of the hidden symptoms but 3.2% of its findings are false and it names only the symptom node. Confirmed slice analysis detects 99.9% of them with no false finding; shrinking a whole-team witness slice by slice returns an exact minimal slice for 99.2% of them and the combined slicing strategy for 86.3%. These rates depend on team size and on the mix of changes. No language model is run, and we make no claim that any model meets a contract.

CCS Concepts: • **Software and its engineering** → **Model checking**; • **Computing methodologies** → **Multi-agent systems**.

Additional Key Words and Phrases: Multi-agent systems, large language models, assume-guarantee reasoning, contracts, compositional verification, fault localization

1 Introduction

Large language model (LLM) agents are now composed into teams that do software work. MetaGPT assigns the roles of a software company to agents that exchange documents in a fixed order [26]; ChatDev runs a chain of design, coding, review and testing phases [50]; AgentCoder lets a programmer agent and a test-designer agent work in parallel and runs the tests with a script [29]; Agentless and MAGIS split issue resolution into localization, repair and validation stages [55, 64]. Surveys list dozens of such designs [20, 24, 28], and frameworks such as AutoGen make it easy to wire agents into new topologies [63].

These teams are engineered artifacts, so we should be able to ask an engineering question about them before they run: *is the composition justified?* If every agent does what its contract says, does the team deliver what the team promises? Current practice answers other questions. Benchmarks score the final output of runs [32, 43, 65]. Guardrails and runtime enforcement check one agent's inputs, outputs or actions [30, 51, 58], and security benchmarks probe one agent against injected content [16, 22, 53]. Failure studies collect traces and attribute a failure to an agent and a step after the fact [8, 67]. Cemri et al. report that many failures of multi-agent LLM systems come from inter-agent misalignment and from verification steps that do not verify [8]. These are properties of relationships among agents.

Consider a release gate that ships a change when either the code reviewer or the security tester approves. Each agent can meet its contract, and the gate's policy is legal, yet the team can ship code

Author's Contact Information: Anonymous Authors.

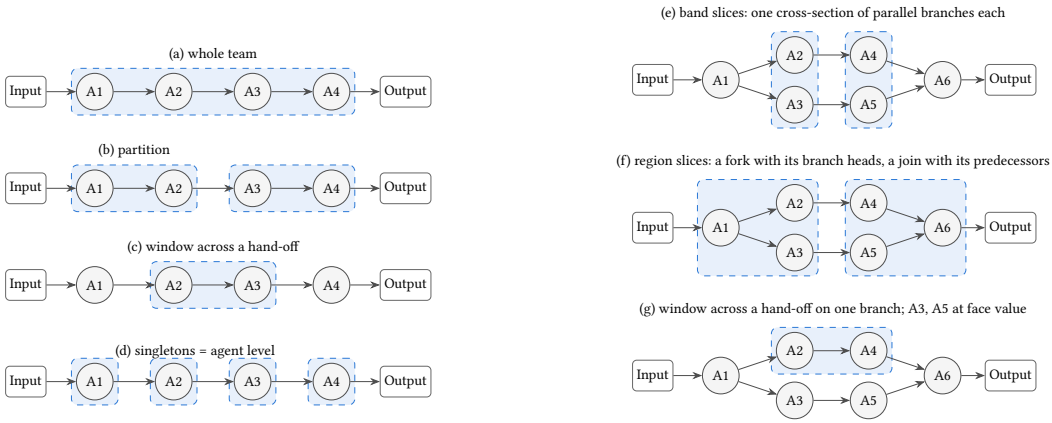


Fig. 1. Slices of a chain (a–d, left) and of a team with a fork and a join (e–g, right). Slicing is defined on the team graph, not on a sequence: the same notions apply when agents branch and re-merge. A slice is a convex set of agents analyzed exactly while the rest of the team is taken at face value. Singleton slices (d) give agent-level analysis; the whole team (a) gives exact monolithic analysis. Slices in between expose and localize defects that live in hand-offs, joins and shared models. On the graph, slices may follow a cross-section of parallel branches (e), wrap a fork or a join (f), or follow one branch (g); a slice such as those in (e) need not be connected.

that was never security tested. Inspecting any single agent shows nothing wrong. A whole-team analysis shows that a property can be missing at the output, but not *which* agents let it through.

Assume-guarantee reasoning gives the vocabulary: each component assumes something about its inputs and guarantees something about its outputs, and a composition is justified when every assumption is discharged [1, 5, 34, 47]. Applied to one agent at a time, with every other agent assumed to keep its contract, the check is not path-sensitive. It credits a fact to a consumer if some upstream agent claims it, even if a join may drop that branch, and it never sees two agents fail together. Classical whole-program analyses fix part of this: we show that a must-analysis in the style of monotone data-flow frameworks [36, 37], run once per failure set, detects most such defects. But it over-approximates, so it raises false alarms, and it names only the node where the symptom appears.

This paper. We propose *agent slicing*: analyze *parts of a team* exactly and the rest at face value. A slice is a set of agents that is convex in the team graph (every path between two members stays inside); it need not be connected. Within a slice, joins, routing choices and model failures are explored exactly; outside, every component is assumed to keep its contract. Face value is an approximation in both directions, so a slice can report a defect the team does not have. We therefore *confirm* each slice finding by lifting its witness to a run of the whole team. The size of the smallest slice that confirms a defect is its *depth*: depth one means the defect is visible to an agent-by-agent check, depth two or more means it is hidden. The smallest confirming slices *localize* the defect: they name the agents whose relationship is wrong. Fig. 1 shows the idea. It is not limited to a sequence of agents: the left side slices a chain, while the right side slices a team graph with a fork and a join, where a slice can follow a branch, cut across parallel branches, or wrap a fork or join, so that the same definitions cover branching teams.

We study three research questions.

RQ1, Formal foundation and expressiveness. *Can an agent slice be defined as a first-class contract such that slice-level reasoning is sound and compositional, recovers agent-level analysis as a special case, and is strictly more expressive than it?*

RQ2, Detection and localization power. *How many blind spots does slice-level analysis reveal that agent-level analysis cannot, and how precisely does it localize their root causes, compared with agent-level and whole-team analysis?*

RQ3, Cost, slicing strategy, and evolution. *What does slice-level analysis cost, how do slicing strategy and slice size trade detection power against cost and false alarms, and how much re-verification does slicing save when a team evolves?*

Contributions.

- A calculus of graph-shaped agent teams with fork, switch, join policies, conditional guarantees and model failures, with an exact semantics (Section 2).
- *Slices as analyzable components*: convex slices, a hybrid semantics whose approximation we characterize, confirmation by lifting, per-exit derived slice contracts, emergent obligations as predicates over witnesses, and slicings with an explicit acyclic-quotient condition (Section 3).
- The notions of *depth* and *minimal confirming slice*, and a taxonomy of blind spots (A1–A4, S1–S6) defined by the mechanism of their witness.
- Eight results (Proposition 1, Theorems 2–5, Proposition 6, Theorems 7 and 8) with paper proofs, some steps of which are sketched (Appendix A). Per-node steps are SMT obligations, and every result with a computational statement is checked against brute-force oracles on small teams.
- An implementation, three benchmarks (injected defects, defects induced by random engineering changes, encodings of nine published designs), seven baselines, and a replication package that regenerates every number in the paper.

Main results. On teams with defects induced by random engineering changes (8–32 nodes), 36.1% of the 3,863 symptoms have depth at least two. That an agent-by-agent check finds none of these is true by definition of depth; the measured part is how many there are, and that for 210 of the 1,345 defective teams (190 of them unsafe) such a check reports nothing at all. A global must-analysis finds 100.0% of the hidden symptoms in linear time, but 3.2% of its findings are false and it localizes only to the symptom node. Confirmed slice analysis finds 99.9% of them with no false finding. For localization, the best pipeline in our data is: detect (whole team or must-analysis), confirm, then shrink the witness slice by slice; it returns an exact minimal confirming slice for 99.2% of hidden symptoms at about 29 slice checks each. The combined slicing strategy, which needs no whole-team run, is exact for 86.3%, and the deviation set of a whole-team witness for 1.0%. The hidden share and the strategy's exactness change with team size (Section 4.4). Depth equals the number of hand-offs between the interacting agents plus two for the injected defect families, independently of host size. Small slices scale to thousands of agents; on choice-rich teams both large slices and whole-team checking hit time limits.

Scope. We ask whether a composition is justified *given the declared contracts* of its parts. We do not claim that any LLM satisfies a contract, that our contracts describe the encoded systems faithfully, or that any deployed team became safer. Compositional certification with declared contracts (Theorem 2b) is proved only without model failures; with failures, composition uses exact boundary behaviors. Confirming a finding requires a search over runs of the whole team, so confirmed slice analysis is not a local check. No language model is run.

The rest of the paper is organized as follows. Section 2 defines teams and their semantics and introduces the running example. Section 3 presents agent slicing and its theory. Section 4 answers

Table 1. Notation

Symbol	Meaning
$\mathcal{P}$	properties (facts about the work product)
Φ, Φ_c	goals of the team; critical goals
f	fault budget: how many models may fail in one run
N, E, ι, ω	nodes, edges, input node, output node
req, *gua*, *decl*	requirements, guarantees, declared evidence of a node
$orc(n)$	the model (oracle) an agent uses
$\rho = (W, C, F)$	a run: switch choices, contributor sets, failed oracles
$c(v), o(v)$	content (what v receives) and output of v in a run
$(\mathsf{kind}, v, \mathsf{p})$	a symptom key: kind, node, property
$Sym(T)$	all symptoms of team T (exact semantics)
σ	a slice (convex node set)
H_σ	hybrid semantics: σ exact, rest at face value
$\mathcal{F}(\sigma)$	raw findings of σ (keys of H_σ)
$C(\sigma)$	confirmed findings of σ (witness lifts to T)
$depth(\kappa), MMS(\kappa)$	depth; minimal confirming slices of key κ

the research questions. Section 5 discusses related work, and Section 6 concludes. The appendices contain the proofs, the specification grammar, and additional tables.

2 Background: Agent Teams and Their Semantics

This section fixes a small model of agent teams. Table 1 summarizes the notation.

2.1 Agents, Gates and Contracts

A team works on a shared artifact, for example a code change, and builds up facts about it: a design exists, adequate tests exist, the tests pass. We call such facts *properties* and write $\mathcal{P}$ for the finite set of properties a team mentions. A component *requires* some properties before it can do its job, and it *guarantees* others when it does its job. This is design by contract [46] applied to team members.

Properties are semantic claims, not artifacts. "tests" means that an adequate test suite exists, not that a file named `tests` was produced. When an agent's model fails, the agent may still produce an artifact, for example a weak test suite, but the property it claims does not hold. A downstream test runner that requires tests then cannot establish tests_pass, even if it executes and reports success. So the model covers errors of commission (a wrong artifact presented as right) as well as omission, at the level of properties. What the model does not distinguish is what happens next. A symptom at the output says that the team can release without the property being established; it does not say whether the release is a false certification (the runner reports "pass" on weak tests) or a visible failure that a later step would catch (the runner reports "fail" and someone retries). Safety here is partial correctness: a run that never reaches the output produces no symptom, so a pipeline that blocks on a failed check is safe in the model only if the block is modeled as not reaching the output.

Two kinds of components matter. An *agent* is driven by a language model. Its guarantee is *conditional*: it holds only when the agent's requirements are met and its model (its *oracle*) does not fail. A *gate* is a deterministic program, such as a test runner or a schema check. A gate's guarantee is conditional on its requirements but never on a model.

An agent may also *declare* evidence for itself, for example "I read the specification". Declared evidence counts when we decide whether that agent can act, but it never flows to other components. Semantically, declaring p is the same as removing p from the agent's own requirements; we keep the notion because it records a common documentation practice and makes its locality explicit.

Definition 1 (Team). *A* team *is a tuple* $T = (N, E, \iota, \omega, \kappa, req, gua, decl, orc, rt, jn)$ *with goals* $\Phi \subseteq \mathcal{P}$*, critical goals* $\Phi_c \subseteq \Phi$*, and a fault budget* $f \in \mathbb{N}$*, where:*

- (N, E) *is a finite directed acyclic graph with a unique source* ι *(the input) and a unique sink* ω *(the output), and every node lies on a path from* ι *to* ω*;*
- $\kappa(n) \in \{\mathsf{agent}, \mathsf{gate}\}$*;* ι *and* ω *are gates;*
- $req(n), gua(n), decl(n) \subseteq \mathcal{P}$ *are the requirements, guarantees and declared evidence of* n*;* $gua(\iota)$ *are the input facts, and* $req(\omega) = \Phi$*;*
- $orc(n)$ *names the oracle of each agent; agents may share an oracle;*
- $rt(n) \in \{\mathsf{fork}, \mathsf{switch}\}$ *says whether* n *delivers to all successors or to exactly one;*
- $jn(n) \in \{\mathsf{all}\} \cup \{1, \ldots, |pred(n)|\}$ *is the join policy of a node with several predecessors;* any *is the quorum* 1*.*

We write $pred(n)$ and $succ(n)$ for direct predecessors and successors, and $anc(n)$ and $desc(n)$ for proper ancestors and descendants; anc^* and $desc^*$ include the node itself. A switch with one successor is a fork.

2.2 Runs

A run resolves every choice the team leaves open: which successor a switch picks, which delivered inputs a quorum or any join actually uses, and which oracles fail.

Definition 2 (Run). *A* run $\rho = (W, C, F)$ *chooses a successor* $W(n)$ *for each switch, a contributor set* $C(n)$ *for each node with several predecessors, and a set* F *of failed oracles with* $|F| \leq f$*. A run is* fault-free *if* $F = \emptyset$*. In topological order:*

(1) ι *is reached, with content* $c(\iota) = \emptyset$ *and output* $o(\iota) = gua(\iota)$*;*
(2) *an edge* (u, v) *is* delivered *if* u *is reached and* u *is a fork or* $W(u) = v$*;* $D(v)$ *is the set of delivered in-edges of* v*;*
(3) v *is reached if* $D(v)$ *meets its policy: all in-edges for all (and for a single in-edge), at least* q *for quorum* q*; then* $C(v)$ *is all in-edges for all, and a subset of* $D(v)$ *with* $|C(v)| \geq q$ *for quorum* q*;*
(4) *the content is* $c(v) = \bigcup_{(u,v) \in C(v)} o(u)$*;*
(5) v applies *if* $req(v) \subseteq c(v) \cup decl(v)$ *and* v *is a gate or* $orc(v) \notin F$*;*
(6) *the output is* $o(v) = c(v) \cup gua(v)$ *if* v *applies, and* $o(v) = c(v)$ *otherwise.*

Properties flow along the contributing lineage. A join that uses one branch forgets what the other branch established; this is what makes any and quorum joins interesting.

2.3 Symptoms and Safety

A team can go wrong in three observable ways: a component acts without a property it requires, the output misses a goal, or the output misses a critical goal when up to f oracles fail. We name each by a *key*, a triple of kind, node and property.

Definition 3 (Symptoms, safety). *The* symptoms $Sym(T)$ *of a team are the keys*

- $(\mathsf{assume}, v, \mathsf{p})$*: some fault-free run reaches* $v \neq \omega$ *with* $p \in req(v) \setminus (c(v) \cup decl(v))$*;*
- $(\mathsf{goal}, \omega, \varphi)$*: some fault-free run reaches* ω *with* $\varphi \in \Phi \setminus c(\omega)$*;*
- $(\mathsf{robust}, \omega, \varphi)$*:* $(\mathsf{goal}, \omega, \varphi) \notin Sym(T)$*, and some run with* $|F| \leq f$ *reaches* ω *with* $\varphi \in \Phi_c \setminus c(\omega)$*.*

T *is* safe *if* $Sym(T)$ *contains no goal or robust key, and* clean *if* $Sym(T) = \emptyset$*.*

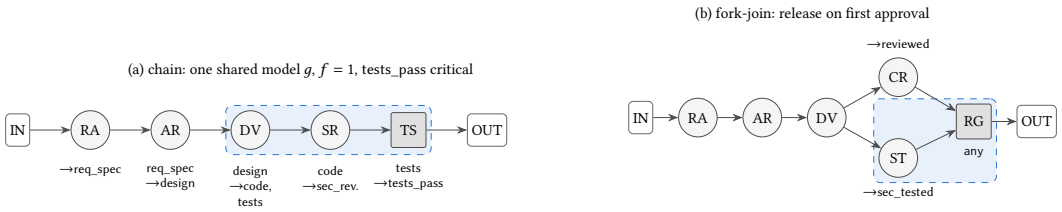


Fig. 2. Running example. Circles are agents, squares are gates. Labels show requirements and (→) guarantees. Agent-level analysis reports nothing for either team. Shaded: minimal confirming slices, {DV, SR, TS} for the chain (a robust loss of tests_pass) and {ST, RG} (one of two) for the fork-join team.

Two remarks on scope. Safety is a partial-correctness notion: a run in which the output is never reached (for example, an all-join behind a switch) produces no symptom. And under failures we check only critical goals at the output; an agent acting without its requirements because an upstream model failed is the expected consequence of the failure, not a design defect.

2.4 From Models to Runs

The guarantee we want is about all behaviors that respect the contracts. In such a behavior a component may know more than its content: an agent may happen to see a fact nobody guaranteed.

Definition 4 (Contract-respecting model). *A* model *of T is a run (W, C, F) together with evidence sets $E(v) \supseteq c(v) \cup decl(v)$ for reached nodes, where contents are computed as in Definition 2 except that v applies if $req(v) \subseteq E(v)$ and its oracle has not failed. Its keys are those of Definition 3 with $E(v)$ in place of $c(v) \cup decl(v)$ (and $E(\omega)$ in place of $c(\omega)$).*

Models differ from runs only by extra evidence. This is a modeling choice: we let a component know more than it was told, but not less. The following lemmas show that runs, and only a few of them, suffice.

Lemma 1 (Worst case). *A key is produced by some model if and only if it is produced by some run.*

Lemma 2 (Minimal choices). *Content is monotone in the contributor sets and antitone in the failure set. To decide $Sym(T)$ it suffices to enumerate runs whose quorum joins use exactly q contributors and whose failure sets have size* $\min(f, |Orc|)$.

The idea behind Lemma 1 is that more evidence can only make more components apply, which only adds properties downstream; the minimal evidence $c(v) \cup decl(v)$ is the worst case. Proofs are in Appendix A.

2.5 Running Example

We use two small software teams throughout (Fig. 2).

Chain. A requirements analyst (RA) writes a specification; an architect (AR) turns it into a design; a developer (DV) writes code and tests; a security reviewer (SR) reviews the code; a deterministic test runner (TS) runs the tests and guarantees tests_pass. The goals are tests_pass and sec_reviewed; tests_pass is critical, and one model failure is tolerated ($f = 1$). All four agents use the same model g.

Look at each component alone. AR needs req_spec, which RA guarantees upstream. DV needs design, guaranteed by AR. SR needs code and TS needs tests, both guaranteed by DV. The critical goal is guaranteed by a *gate*, so it seems not to depend on any model. Every assumption is matched; an agent-level check reports nothing. Yet the team is not robust. If model g fails at DV, the tests DV

writes are not adequate, TS cannot establish tests_pass, and the release lacks its critical goal. The defect is neither in DV (it is allowed to fail; $f = 1$) nor in TS (it behaves correctly). It lies in the relationship: *a deterministic gate's critical guarantee rests on a fallible agent's provision.*

Fork-join. The analyst, architect and developer form a chain. The developer's code then forks to a code reviewer (CR), which guarantees reviewed, and a security tester (ST), which guarantees sec_tested. Both feed a release gate (RG) with policy any: release on the first approval. Every requirement is matched and every goal is claimed by an ancestor of the output. Agent-level analysis is silent. But a run in which RG uses only CR's approval releases code that was not security tested, and the converse run releases code that was not reviewed.

3 Agent Slicing

3.1 Overview

The analysis takes a team specification and a *slicing strategy*. The strategy proposes slices. For each slice, the analysis explores the slice's internal behaviors exactly while taking the rest of the team at face value, and records every symptom it sees with a witness. Each witness is then *lifted*: the analysis searches for a run of the whole team that agrees with the witness inside the slice and shows the same symptom. Only confirmed findings are reported, each with its smallest confirming slices, a mechanism class, and the lifted run. Safety is certified separately, either with the slice of all nodes or compositionally with a partition (Theorem 2). As an alternative to fixed strategies, a whole-team witness can be *shrunk* slice by slice until no member can be removed.

3.2 Slices

A slice should be a well-defined sub-computation: work that enters it should not leave it and come back. Otherwise the slice's behavior would depend on what happens outside in the middle of its own computation.

Definition 5 (Slice). *A* slice *is a nonempty set $\sigma \subseteq N$ that is* convex*: for all $u, w \in \sigma$, every path from u to w lies in σ. Its* entry edges *go from outside into σ; its* exit edges *go from σ to outside. The* hull *$hull(U) = anc^*(U) \cap desc^*(U)$ is the smallest convex superset of $U \subseteq N$.*

Connectedness is not required. In the fork-join team, {CR, ST} is a slice: there is no path between the two. {RA, DV} is not a slice, because the path RA→AR→DV leaves it; its hull is {RA, AR, DV}.

Lemma 3 (Convexity). *If σ is convex, no node outside σ is both an ancestor and a descendant of members. Hence outside predecessors of members have no member ancestor. Contracting σ to one node (all other nodes kept) leaves a DAG if and only if σ is convex.*

3.3 Hybrid Semantics

To analyze a slice in isolation we must say what the rest of the team does. We take it at *face value*: every outside component is assumed to keep its contract and to deliver everywhere.

Definition 6 (Hybrid semantics). *The hybrid semantics H_σ is defined like Definition 2 with these changes. Runs choose only member switches, member contributor sets, and failures $F \subseteq orc(\sigma)$ with $|F| \leq f$. Every* outside *node is reached if it is ι or at least one in-edge is delivered (whatever its policy), all its delivered in-edges contribute, it always applies, and it delivers to all successors. Every entry edge into a member v is delivered and carries the same environment $env_\sigma(v) = \bigcup_{u \in pred(v) \setminus \sigma} o(u)$, the union of the face-value outputs of all outside predecessors. The* raw findings *$\mathcal{F}(\sigma)$ are the keys of Definition 3 evaluated in H_σ at every node, where a robust key requires the goal key of the same property to be absent from $\mathcal{F}(\sigma)$, as in Definition 3. For confirmation (Definition 7) we use the* unsuppressed *robust*

candidates: a robust key is a candidate whenever some run of H_σ with failures misses a critical goal at ω, whether or not the slice also shows the goal key.

By Lemma 3, the outside predecessors of a member are never influenced by members, so $env_\sigma(v)$ is a fixed set. Outside nodes *below* members are recomputed in each run; this is how an internal failure becomes visible at the output. With $\sigma = N$ nothing is outside, H_N is the exact semantics, and $\mathcal{F}(N) = Sym(T)$. With $\sigma = \emptyset$ every node applies; $\mathcal{F}(\emptyset)$ are the requirements and goals that no ancestor claims at all (*static* keys).

Lemma 4 (Face value dominates content). *Let $\sigma \subseteq \sigma'$, let ρ' be a run of $H_{\sigma'}$, and let ρ be the run of H_σ with the same choices on σ and failures $F' \cap orc(\sigma)$. Every node reached in ρ' is reached in ρ, and its content in ρ' is a subset of its content in ρ.*

So, run by run, analyzing more components exactly can only remove content. But H_σ also has runs with no counterpart in the team. An outside all-join is reached on a single input, and an outside switch delivers to both branches, so H_σ over-approximates reachability while it under-approximates missing properties. Its raw findings are therefore neither a may- nor a must-approximation of $Sym(T)$.

Example 1 (A raw false alarm). Let a switch gate s route either to a gate a (which guarantees x) or to the output, and let a gate J with an all-join of ι and a require x; the output joins s and J with any. In the team, J is reached only when s routes to a, and then x is present: $Sym(T) = \emptyset$. In the singleton slice $\{s\}$, J is outside and reached on ι's edge alone, so $(\text{assume}, J, \mathsf{x}) \in \mathcal{F}(\{s\})$.

We therefore never report a raw finding directly.

Definition 7 (Confirmed findings). *A raw finding or robust candidate κ of σ* lifts *if some run ρ of T exhibits κ (for a robust key: misses the property at ω with $1 \le |F| \le f$), and the* replay *of ρ on σ exhibits κ as well. The replay is the run of H_σ that makes ρ's switch and contributor choices at members, fails $F \cap orc(\sigma)$, and, at members that ρ does not reach (but H_σ does), uses all delivered inputs and delivers to all successors if a fork and to none if a switch. The* confirmed findings $C(\sigma)$ *are the keys that lift.*

The definition ties a finding to the slice: the team must show the symptom, *and* the slice must show it for the same internal choices. The implementation first searches team runs with the member switch choices of each recorded witness fixed (up to 64 witnesses per key), and then, if none lifts, searches all runs of the team; each candidate run is replayed in H_σ. The second search is complete; a search that exceeds its budget (200,000 runs) is counted and the key is treated as unconfirmed, which happened 0 times in E5. In the worst case, lifting is as hard as whole-team checking (Theorem 7); in our benchmarks a slice finding needed 7.1 lift searches on average and each took milliseconds.

3.4 Agent-Level Analysis

What does it mean to check a team agent by agent? The natural reading is: check each agent against its own contract, assuming every other component keeps its contract. That is the hybrid semantics of a singleton slice.

Definition 8 (Agent-level analysis). *The raw agent-level findings are $\bigcup_v \mathcal{F}(\{v\})$; the (confirmed) agent-level findings are $AL(T) = \bigcup_v C(\{v\})$.*

Proposition 1 (What agent level sees). *For every node v, $\mathcal{F}(\emptyset) \subseteq \mathcal{F}(\{v\})$, and the keys located at v in $\mathcal{F}(\{v\})$ are exactly the static keys at v. Every key in $\mathcal{F}(\{v\}) \setminus \mathcal{F}(\emptyset)$ has a witness in which v does not apply (its requirements are unclaimed upstream, or its oracle fails) or v is a switch.*

Table 2. Derived and Naive Exit Guarantees in the Running Example (Beyond What the Entries Carry)

Slice	Entries carry	Naive N_σ	Derived G_σ / G^f_σ
DV, SR, TS	req_spec, design	code, tests, sec_rev., tests_pass	G: same; G^f: nothing
SR, TS	+code, tests	sec_rev., tests_pass	G: same; G^f: tests_pass
ST, RG	spec, design, code	sec_tested	G: nothing
CR, RG	same as above	reviewed	G: nothing

The proposition explains the silence in the running example. A singleton slice gives every entry edge the same environment, so a join's own singleton sees the union of its inputs and never the choice it makes. One agent's failure is seen, but never two agents failing together, and a gate never sees where its inputs came from. In the chain, $\mathcal{F}(\{v\}) = \emptyset$ for every v: the singleton {DV} can fail, but TS outside still applies at face value.

3.5 Slice Contracts

A slice is a component, so it should have a contract. Within a slice, a member's requirement may be established by another member, or it may have to come from outside; only the latter belong in the slice's assumptions. And the slice guarantees on an exit only what holds there on every behavior.

Definition 9 (Derived slice contract). *For a slice σ and an environment $X \subseteq \mathcal{P}$ carried by every entry edge:*

- *the* entry assumptions A_σ *are the pairs (v, p) with $v \in \sigma$ and $p \in req(v)$ such that some fault-free internal run with empty environment reaches v without p in $c(v) \cup decl(v)$;*
- *the* exit guarantee $G_{\sigma,e}(X)$ *of an exit edge e is the intersection, over the fault-free internal runs that deliver e, of the output crossing e; $G^f_{\sigma,e}(X)$ is the same over runs with $|F| \le f$; the uniform guarantee is $G_\sigma(X) = \bigcap_e G_{\sigma,e}(X)$;*
- *the* naive guarantee $N_{\sigma,e}(X)$ *is what composing member contracts at face value promises on e: X plus every guarantee of a member that is an ancestor-or-self of the exit's source.*

The gap between naive and derived guarantees is what a slice adds. Each confirmed finding witnesses such a gap, and its witness tells *why* a claimed property did not arrive. For a key κ about property p at node x, with confirming witness ρ on σ, each claimer u of p among the ancestors of x is in one of four states: *route* (u is unreached because a switch chose another branch), *drop* (u applied but a join did not take its output into x's lineage), *cond* (u did not apply because a requirement was missing), or *fault* (u did not apply because its oracle failed). The *emergent obligations* X_σ are the slice-level conditions that exclude these states, and the taxonomy (Table 3) classifies findings by the state that occurs. Joint coverage excludes *drop*, non-bypass excludes *route*, conditional backing excludes *cond* under failures, and oracle independence excludes two *fault* claimers on one oracle. No single member can carry these obligations: each speaks about two or more components.

Table 2 shows derived contracts for the running example. For {DV, SR, TS} the fault-free guarantee equals the naive one, but the robust guarantee loses code, tests, sec_reviewed and tests_pass. For {SR, TS}, with tests supplied from outside, the robust guarantee keeps tests_pass, so the slice that *explains* the defect must contain DV. For {ST, RG} the naive guarantee contains sec_tested but the derived one does not: joint coverage fails at RG.

Definition 10 (Declared contract, exactness). *A* declared contract *of σ is a pair (A^d, G^d). It is* valid *if, when every entry carries A^d, every member's requirements hold in every fault-free internal run and $G^d \subseteq G_{\sigma,e}(A^d)$ for every exit e. A pair (A, G) is* exact *for exit e if, for every environment X, $G_{\sigma,e}(X) = X \cup G$ when $A \subseteq X$ and $G_{\sigma,e}(X) = X$ otherwise.*

Definition 11 (Boundary behavior). *For a slice σ and a failure set F, Beh_σ maps each delivery pattern of the entry edges and each assignment of contents to entry edges to the set of possible outcomes: which exits are delivered, with what content, and which keys arise at members. Contents are tracked symbolically: a member's content is the union of the contents of the entry edges in its* flow *plus what members add.*

Theorem 2 (Slice abstraction). (a) *Replacing a convex slice σ by a macro component whose behavior is Beh_σ (evaluated with the run's global failure set) leaves every assume and goal key of T unchanged, and leaves the set of properties missing at ω under each failure set unchanged; hence $Sym(T)$ is unchanged.*

(b) *Let $f = 0$ and let every run of T deliver either all or none of σ's entry edges (as for single-entry slices). If (A^d, G^d) is valid and the team in which σ is replaced by a component that, when its entries carry A^d, delivers on any subset of its exits (possibly none) with content at least $A^d \cup G^d$, and otherwise delivers on any subset with any content, has no goal key, then T has no goal key.*

(c) *An exit whose guarantee is g if r_1 or r_2 enters (two members $r_1 \to g$ and $r_2 \to g$ feeding one exit node) has no exact single pair (A, G), for any distinct r_1, r_2, g.*

(d) *Let $\{\sigma_1, \ldots, \sigma_m\}$ partition N (including ι and ω) into convex parts whose quotient graph is acyclic. Composing the parts' boundary behaviors in the quotient's topological order, with one global failure set per run, yields $Sym(T)$; robust keys are decided after composition. Convex parts alone do not suffice: $\{a, d\}$ and $\{b, c\}$ with edges $a \to b$ and $c \to d$ are convex and their quotient has a cycle. With a cyclic quotient, valid flat part contracts can certify an unsafe team.*

Proof idea. (a) and (d) hold because a part's outcomes are a function of what crosses its entries and of the global failure set, and the symbolic flow makes the summary independent of the concrete entry contents. Failure sets must be global because one oracle can serve several parts; this is exactly what makes shared-model defects (S4) visible to composition. Robust keys depend on the absence of a goal key anywhere, so they are decided after composition. (b) follows from (a) because the declared component over-approximates Beh_σ under the stated delivery condition. (c) is the SMT obligation C2 (Table 5), stated over an uninterpreted sort of properties: exactness on the three environments $\emptyset$, $\{r_1\}$ and $\{r_2\}$ is already contradictory. For the cyclic example add $\iota \to a$, $\iota \to c$, $b \to \omega$, $d \to \omega$ and goals $\{x, y\}$; let a and d require y and guarantee x, and b and c require x and guarantee y. The part contracts "$\{a, d\}$: assume y, guarantee x" and "$\{b, c\}$: assume x, guarantee y" are valid and discharge each other, but neither x nor y is ever established and the output misses both goals.

3.6 Checking Slices and Soundness

Algorithm 1 is the slice-level checker. For each slice it enumerates the minimal internal runs (Lemma 2) by depth-first search over member choices, propagates each run's effect to outside descendants (stopping where content equals face-value content), collects every distinct witness per key, and confirms keys by lifting. Findings of all slices are merged by key (Proposition 6), and each key is localized to its inclusion-minimal confirming slices.

Algorithm 1 Confirmed slice-level checking and localization

Require: team T, family of slices $\mathcal{S}$, run budget B

$M \leftarrow$ empty map from keys to (slice, witness) pairs
for all static keys $\kappa \in \mathcal{F}(\emptyset)$ **do**
  if some run of T reaches κ's node **then** $M[\kappa] \leftarrow \{(\bot, \cdot)\}$
end for
for all $\sigma \in \mathcal{S}$ **do**
  for all $F \in \{\emptyset\} \cup \{F \subseteq orc(\sigma) : |F| = \min(f, |orc(\sigma)|)\}$ **do**
    for all minimal internal run ρ of σ with failures F (DFS; abort after B) **do**
      evaluate members; propagate to outside descendants
      record each key of H_σ with witness ρ
    end for
  end for
  for all recorded keys κ (not static) **do**
    if some witness of κ lifts (Def. 7) **then** add (σ, ρ) to $M[\kappa]$
  end for
end for
for all keys κ in M **do**
  $MMS(\kappa) \leftarrow$ inclusion-minimal slices in $M[\kappa]$, ranked by size, then by number of choice points
  classify κ by the claimer states of its witness (Table 3)
end for
return M, *MMS*, classes

Theorem 3 (Soundness). (a) *If the family contains N, or if certification composes a partition with an acyclic quotient (Theorem 2d), the verdict "safe" implies that T is safe, and each counterexample is a run of T.*

(b) *Every confirmed finding is a symptom of T: $\kappa \in C(\sigma)$ implies $\kappa \in Sym(T)$, or, for a robust key, that T loses the property at ω under at most f failures (and $Sym(T)$ then contains the robust key or the stronger goal key).*

(c) *Raw findings can be false alarms (Example 1).*

3.7 A Taxonomy of Blind Spots

Table 3 lists the blind-spot classes. Each is defined by the claimer states in a confirming witness (previous subsection), so the class of a finding is computed, not guessed. The first five classes are visible at agent level; S2–S6 involve a relationship among components.

S5 is visible at the switch: the switch's singleton slice explores its choice, so its depth is one. S6b shows that "visible" is not "explained": agent-level analysis reports the unmet root assumption but never the goal lost several hand-offs later.

3.8 Depth and Localization

Definition 12 (Depth, minimal confirming slices). *A slice σ* confirms *a key κ if $\kappa \in C(\sigma)$. A confirmed static key has depth* 1 *and is localized at its node. Otherwise the* depth *of κ is the minimum size of a confirming slice, and $MMS(\kappa)$ is the set of inclusion-minimal confirming slices. A symptom is* hidden at agent level *if its depth is at least* 2.

Table 3. Blind-Spot Classes, Defined by the Claimer States of a Confirming Witness. "AL": Visible to Agent-Level Analysis

Class	Mechanism (witness states)	Agent-level visibility
A1	unowned goal: no component claims a goal (static)	yes
A2	missing provider: a requirement no ancestor claims (static)	yes
A3	ordering: the only claimer is not an ancestor of the consumer (static)	yes
A4	single point of failure: the one fallible claimer of a critical goal is in state *fault*	yes
S1	interface gap: provided in a different view (spec@raw vs. spec@summary)	yes
S2	joint-coverage gap: goal claimers in state *drop* at an any/quorum join	no
S3	path-merge dilution: as S2 for a fact consumed after the join	no
S4	correlated evidence: two or more claimers of a critical goal in state *fault* on one oracle	no
S5	bypass: a claimer in state *route* (a switch routes around it)	at switch
S6a	conditional collapse: a gate's critical claim in state *cond* because an agent provision is in state *fault*	no
S6b	conditional chain: a goal claimer in state *cond* because a root assumption fails; the root is visible, the lost goal is not	no

Depth and minimal slices are semantic: they are defined over confirmed findings, so a slice whose face-value world shows a symptom the team does not have, or shows a true symptom for a reason the team does not have, does not count. In the running example, the chain's robust loss of tests_pass has depth 3 with the unique minimal slice {DV, SR, TS}; SR is included because a slice must be convex. Each goal loss of the fork-join team has depth 2 with 2 minimal slices, {CR, RG} and {ST, RG}: the join together with the branch it may take alone.

Theorem 4 (Separation). (a) *For each class S2, S3, S4, S6a and S6b there is a family of teams, parametric in the number $\ell \geq 0$ of hand-offs between the interacting components (and, for S2 and S3, in the number $m \geq 2$ of branches), in which AL(T) contains no key of the defect (for S6b: not the goal key), the team is unsafe or unclean, and the defect has depth exactly 2 (S2, S3) or exactly $\ell + 2$ (S4, S6a, S6b).*

(b) *Let a* view *of node v be its contract, kind, oracle, policies and the set of properties claimed by its ancestors. Call an analysis* view-disjunctive *if it flags a team if and only if some node's view satisfies a fixed predicate. For S2, S4 and S6a there are safe teams whose views together contain every view of an unsafe team; so no view-disjunctive analysis is both sound and free of false alarms on these classes.*

(c) *Recovery: every confirmed agent-level finding is confirmed by every slice that contains its singleton, and every static key at a node v is a raw finding of every slice in which v is reached in some run.*

Part (b) is a limit on agent-local methods, but only on those that decide one node at a time. An analysis that combines views, such as a check that two claimers share an oracle, escapes it; so does a global must-analysis, which propagates facts along paths. We compare against both in Section 4. Part (c) follows from the monotonicity of confirmed findings (Theorem 5c). It does not hold for raw findings: a raw agent-level finding can disappear in a larger slice, because the larger slice

shows that the singleton's witness was impossible. Together, (a) and (c) make confirmed slice-level analysis strictly more expressive than confirmed agent-level analysis.

Theorem 5 (Localization). (a) *If $\kappa \in Sym(T)$ then $MMS(\kappa) \neq \emptyset$; in particular N confirms κ.*

(b) *$MMS(\kappa)$ need not be a single slice. It is a single slice if and only if the intersection of the minimal confirming slices is nonempty and confirms κ.*

(c) *Confirmed findings are monotone: if $\sigma \subseteq \sigma'$ are slices, then $C(\sigma) \subseteq C(\sigma')$. Hence the confirming slices of a key are closed upward, and $depth(\kappa) \leq k$ iff some slice of size at most k confirms κ.*

(d) *Raw findings are not monotone: there are teams and slices $\sigma \subset \sigma'$ with $\kappa \in \mathcal{F}(\sigma) \setminus \mathcal{F}(\sigma')$.*

For (d), let a switch s route either to a gate x or to the output, let x join s and a provider y of the goal g with any, and let the output join s and x with all. The team is clean. In $\sigma = \{s, x, y\}$ the output is outside and reached on one input; the run in which s routes to x and x uses only s's input misses g. In $\sigma \cup \{\omega\}$ that run no longer reaches the output. Larger slices therefore remove raw false alarms as well as reveal deeper defects; confirmation makes the picture monotone.

Proposition 6 (Overlap). *Keys do not depend on the slice that produced them, so findings of overlapping slices merge by key without double counting; a slice only reports the presence of keys, so findings of different slices cannot contradict each other. The family of all convex slices of size at most k confirms every key of depth at most k.*

3.9 Slicing Strategies

Definition 13 (Slicing). *A* slicing *is a family of slices. A* partition slicing *partitions N into slices whose quotient graph is acyclic. A* covering slicing *is any family; its slices may overlap.*

We compare these strategies. *Singleton* is agent level. *Whole* is the slice N. *Window-k* takes, for each node, the hull of the node and its descendants within $k-1$ hops; on a chain these are the intervals of length k (Fig. 1c). *Band-w* takes the hull of w consecutive longest-path levels, a cross-section of parallel branches (Fig. 1e). *Region* takes single-entry single-exit regions: each fork or switch with its immediate post-dominator, each join with its immediate dominator, and each join with its direct predecessors [33, 39]. *Goal-directed* is the analogue of a backward slice [60]: for each requirement (v, p), the hull of v, the ancestors that claim p, and the ancestors that claim those claimers' requirements. *Exhaustive-k* takes every convex slice of size at most k. *SL* is singleton $\cup$ window-2 $\cup$ region $\cup$ goal-directed. *Shrink* starts from the whole team (or a whole-team witness) and greedily removes one member at a time while the slice stays convex and still confirms the key, which yields a slice that is minimal under single removals. The analogy with program slicing is loose: only goal-directed slices have a slicing criterion, and we prove no projection property for them.

3.10 Complexity

Theorem 7 (Complexity). (a) *Deciding whether an assume or goal key is in $Sym(T)$ is NP-complete, already for teams of gates only, without forks, and with $f = 0$. Deciding whether a robust key is in $Sym(T)$ is in D^{p} and both NP-hard and coNP-hard. Deciding safety is coNP-complete. Deciding whether a node is reached in some run is NP-complete. Deciding $depth(\kappa) \leq k$ for an assume or goal key is NP-complete.*

(b) *Computing $\mathcal{F}(\sigma)$ takes $O(R_\sigma \cdot (|N| + |E|) \cdot \lceil |\mathcal{P}|/w \rceil + R'_\sigma)$ time, where R_σ is the number of minimal internal runs, w the machine word size, and R'_σ the (quadratic) cost of filtering minimal contributor sets. R_σ is bounded by the product of member switch fan-outs, binomial coefficients*

of member quorum joins, and $1 + \binom{|orc(\sigma)|}{f}$*, so it is polynomial when the member choice points and* f *are bounded. Confirming a finding can cost as much as whole-team checking.*

(c) *A team can have* $2^{\Theta(|N|)}$ *convex slices; all convex slices of size at most* k *are enumerated with* $O(|N|^k)$ *hull tests.*

(d) *Singleton, window, band and region strategies produce* $O(|N|)$ *slices, computable in* $O(|N|(|N|+|E|))$ *total time; goal-directed produces at most* $\sum_v |req(v)|$ *slices.*

The hardness of assume and goal keys uses a CNF encoding: one switch per variable chooses a branch that provides t_i or f_i, the branches merge with any, and one gate per clause requires the facts that falsify all its literals and guarantees bad [13]. The output requires bad, so a run misses it exactly when the formula is satisfiable. For robust keys, add an agent providing a token after the variable gadgets and make every clause gate require the token and guarantee a critical goal c; failing the agent's oracle always loses c, and fault-free runs lose c iff the formula is satisfiable, so the robust key is present iff the formula is unsatisfiable. Hence robust-key membership is not in NP unless NP = coNP. Safety stays coNP-complete because "some goal or robust key exists" is equivalent to "some fault-free run misses a goal or some run with at most f failures misses a critical goal". Because reachability is NP-hard, Definition 1 requires only that every node lies on an input-output path; a node that no run reaches produces no confirmed finding.

3.11 Incremental Recertification

Teams change: a prompt is rewritten, a model is swapped, a join policy is relaxed. Slices make re-verification local.

Theorem 8 (Incremental recertification). *Let a change modify node* a*, and let* Δ *be the symmetric difference of* a*'s old and new guarantees. The dynamic raw findings* $\mathcal{F}(\sigma) \setminus \mathcal{F}(\emptyset)$ *of a slice* σ *that is convex in both versions are unchanged unless* σ *intersects Dep, where*

- $Dep = \{a\}$ *for a change of oracle, kind, routing or join policy;*
- $Dep = anc^*(a)$ *for a change of* $req(a)$ *or* $decl(a)$*;*
- $Dep = anc^*(Y)$ *with* $Y = \{a\} \cup \{y \in desc(a) : req(y) \cap \Delta \neq \emptyset\}$ *for a change of* $gua(a)$*.*

For an edge change (u, v)*, let* Δ *be* u*'s face-value output in either version, and* Y *be* $\{u, v\}$ *plus the descendants of* v *in either version that require a property of* Δ*; then Dep is* $anc^*(Y)$ *taken in both versions. Static findings are recomputed in linear time; slices that lose convexity, and slices that a strategy newly generates, are evaluated from scratch. The theorem concerns raw findings; confirmation depends on the whole team, so stored findings are re-confirmed after each change (one lift search per finding).*

Proof idea. An outside node is taken at face value, so its oracle, kind, routing and policy are invisible to H_σ. A requirement matters only at its node, which affects σ's dynamic findings only if the node is a member or lies below one. A guarantee change alters contents only in the bits of Δ, and those bits matter only at nodes that require them (the output requires the goals).

4 Evaluation

4.1 Implementation

The library `slicing` (Python) implements the team model with property sets as bit masks, the exact and hybrid semantics, lifting and confirmation, slice enumeration and the strategies, derived and declared contracts, partition summaries (Theorem 2d), the checkers and baselines, the classification of findings, incremental recertification, a topology generator, a defect injector, and a mutation operator set. Teams are written in a small JSON specification language (Appendix B). Three oracles

Table 4. Planned Hypotheses and Their Outcome

Hypothesis	Statement	Outcome
H1.1	The SMT obligations have their expected validity or countermodel outcomes, and proof mutants are rejected.	Supported.
H1.2	The implementation agrees with brute-force semantic oracles.	Supported on the small teams checked.
H2.1	Slice-level analysis detects relational defects missed by agent-level analysis.	Holds by construction for the injected hidden classes (Theorem 4a); reported as counts.
H2.2	Slicing localizes defects exactly more often than the whole-team and random-slice baselines.	Supported against random slices and the whole-team witness deviation set; not against whole-team detection followed by shrinking.
H2.3	At most 5% of clean or decoy teams raise a false alarm.	Supported: no alarms on these benchmarks.
H3.1	Slicing strategies have polynomial empirical scaling, whereas whole-team checking does not.	Not supported: behavior depends on topology and choice points.
H3.2	Recall saturates as the maximum slice size k increases.	Follows from the injected gadget sizes; E4b varies hand-off distance and host size.
H3.3	Incremental recertification re-verifies fewer than half of the slices.	Supported in aggregate on non-chain topologies; wall-time savings are small.

serve as independent ground truth for conformance testing. The *model oracle* enumerates contract-respecting models (Definition 4) with all switch choices, all admissible contributor sets, all failure sets with $|F| \leq f$, and every evidence assignment between the realized minimum and that minimum plus the requirements and goals of the node (the only properties evidence can affect). The *replay oracle* enumerates all runs of H_σ and replays each with an evaluator that shares no code with the checker's enumerator. The *run oracle* enumerates every run of T and decides confirmed findings directly from Definition 7, without the lifting search. The replay and run oracles share one function with the implementation: the evaluator that replays a run in H_σ, which the lifting search also uses to verify candidates; they share no code with the checker's enumerator or with the lifting search. Proof obligations use Z3 [15] version 5.3.3.

4.2 Experimental Setup

An experiment plan with hypotheses, metrics, tests and decision rules was written before the first experiment and is in the replication package, with a deviation log. The plan was not registered with an external service, so its timing cannot be verified independently. The revision after the first review round changed the method (confirmation), added baselines and benchmarks, and replaced some analyses; all changes are logged and summarized in Section 4.7.

Table 4 lists the outcome of each planned hypothesis; E4b and E8 were added after the first review round and are exploratory.

Benchmarks. We use three. *(i) Mutation benchmark (E8).* The generator builds clean teams in seven topologies: chain, tree, fork-join, layered DAG, series-parallel, random DAG, and *mixed* (series-parallel with switches and any-joins). It assigns contracts in topological order, drawing each requirement from a sound lower bound on the node's content, so teams are clean by construction (the exact semantics confirms this). One goal is critical and $f = 1$. Each team then receives one to three random engineering changes (retried until that many apply): relax a join from all to a quorum, let two agents share a model, drop a guarantee, turn a gate into an agent, add a requirement, turn a fork into a switch, or remove an edge. Whatever symptoms result are the ground truth, computed with the exact semantics and cross-checked with the replay oracle on teams of up to 12 nodes. Neither the class nor the depth of a defect is chosen by us. We build 1,680 teams (7 topologies; 6, 10, 20 and 30 inner nodes; 20 seeds; 1–3 changes). Symptoms deeper than the enumeration bound (152 keys) count as hidden; their localization is not scored. E8 was added after the first review round and is exploratory. *(ii) Injected benchmark (E2).* The injector adds exactly one defect of a chosen class to a clean team, as a small gadget with fresh property names, and records the class, the root-cause nodes and the symptom keys (verified with the exact semantics). It also builds *clean* teams and four *decoys*: structures that look suspicious but are safe (an any-join whose branches claim all goals; shared-model reviewers backed by a gate; a switch whose both branches provide the fact; a fallible provider backed by an independent gate). For each of 7 topologies, 2 host sizes, 10 seeds and 16 classes we build one team, in a *small* benchmark (7–21 nodes) and a *medium* one (22–43 nodes). Because each gadget is fixed per class and cannot interact with its host, this benchmark tests per-class behavior; its class mix, not nature, decides prevalence, and we report no prevalence from it. *(iii) Published designs (E3)*, encoded with a written protocol (Section 4.4).

Ground truth for depth and localization. On teams of at most 16 nodes we enumerate every convex slice of size at most 6, otherwise at most 5, and compute confirmed findings; the inclusion-minimal confirming slices of a key are its true minimal slices. This uses the same hybrid semantics as the checkers, but confirmation is checked against the independent run oracle (Section 4.3).

Checkers. *B1* is agent-level analysis (confirmed singleton slices); *B1-raw* omits confirmation. *SL* and the single strategies are confirmed slice families; *SL-raw* omits confirmation. *B2* is the exact whole-team checker, which returns a verdict and one witness per key. We localize its findings in three ways: the hull of the witness's contributing lineage (which, as a reviewer observed, always contains the input and is therefore the symptom node's whole ancestor cone); the *deviation set* of the witness (failed agents, agents that did not apply, switches, and quorum joins that used fewer inputs than they received, together with the symptom node), which needs no slices; and *B2+shrink*, which shrinks the whole team slice by slice (Section 3). *B3* draws random convex slices matched to SL in number and size. *B4* is a structural lint (coverage by set difference over all claims, ignoring paths). *B5* is a pattern-based smell detector that flags the shapes of S2–S6 without semantics. *B6* is a global must-analysis: a Kildall-style lower bound on content with union at all-joins and intersection at any/quorum joins, run once without failures and once per failure set of size $\min(f, |Orc|)$, which reports a key wherever the lower bound lacks a required property. It is path-insensitive and linear per failure set; it localizes to the symptom node.

Metrics. A checker *detects* a key if it reports it. On the injected benchmark the pre-registered metric is "reports any ground-truth key"; we also report the defect's primary key (for S6b the lost goal rather than the unmet root assumption), which is what separates the classes. A reported key is *false* if it is not a symptom (a robust key also counts as correct when the team loses the property without failures). Localization uses the top-ranked slice of a key: it is *exact* if it is one of the true inclusion-minimal confirming slices, *minimum* if its size equals the depth, and *precise* if it lies within the hull of the injected root cause. The *size ratio* is its size over $|N|$; *cost* counts slice checks.

Table 5. SMT Proof Obligations (Z3 5.3.3). "Valid": the Negation Is Unsatisfiable; "Countermodel": a Satisfying Model Exists, as Intended

ID	Obligation	Used in	Kind	OK
L1	transfer monotone	Lem. 2, 4	valid	✓
L2	face value dominates	Lem. 4	valid	✓
L3	join subset	Lem. 2, 4	valid	✓
L4	missing antitone	Thm. 4, 5; Lem. 2	valid	✓
L5	evidence monotone	Lem. 1	valid	✓
L7	env uniform	Prop. 1	valid	✓
L8	must sound	B6, generator	valid	✓
L9	meet below choice	B6, generator	valid	✓
L10	fault antitone	Def. 3	valid	✓
L11	failed provides nothing	Def. 2	valid	✓
L12	decl enables	Def. 2	valid	✓
L13	decl local	Def. 2	valid	✓
C1	non distributive	B6 incompleteness	countermodel	✓
C2	no exact single pair	Thm. 2c	valid	✓

Statistics. Where outcomes are deterministic per class (0% or 100% on every seed), we report exact counts and one-sided 95% Clopper–Pearson bounds instead of tests. For localization we compare per-unit exact rates (unit: topology × seed) with one-sided Wilcoxon signed-rank tests [61], Holm correction [25] and Vargha–Delaney $\hat{A}_{12}$ [57], and give 95% cluster-bootstrap intervals [19] that resample hosts (topology, size, seed), because each host is reused across classes and checkers [2].

Environment. All experiments run on one machine (20 CPU cores, Linux, Python 3.12, NumPy, SciPy, Z3), with seeds fixed in the scripts. Each result file records the commit, library versions and wall time. One command regenerates all results, tables, figures and every number in this paper.

4.3 RQ1: Formal Foundation and Expressiveness

Method. We validate the theory in three ways. (i) SMT obligations. The per-node steps of the proofs are formulas over arrays from an uninterpreted sort of properties to Booleans, so each holds for every property universe and every contract; an obligation is valid if its negation is unsatisfiable. These are one-step set identities; they guard the definitions against slips, not the inductions over topological order, which are proved on paper. Six *proof mutants* each change one definition (for example, letting declared evidence flow on) and must break the obligation that depends on it. (ii) Conformance. On 20,000 random "wild" teams with arbitrary contracts, policies, switches and faults (3–7 inner nodes) and on 1,120 small injected teams (4–7 inner nodes before injection), we compare the implementation with the three oracles and check each computational theorem statement directly, for all convex slices of size at most 3 (2 for the run oracle). (iii) The hardness reductions of Theorem 7 on 500 random 3-CNF formulas, and the view-disjunctive constructions of Theorem 4b.

Results. All 14 obligations behave as expected (13 valid steps and 1 intended countermodel; Table 5); the countermodel shows that the conditional transfer does not distribute over intersection, which is why a must-analysis cannot be exact. All 6 of 6 mutants are killed. Table 6 shows 0 mismatching teams in 219,200 team-check pairs, including confirmed findings against the run oracle, the monotonicity of confirmed findings, partition composition, and incremental recertification

Table 6. Conformance Against Brute-Force Oracles: Teams Checked (Wild, Injected) and Teams With Any Mismatch

Check	Wild	Inj.	Mism.
Exact semantics vs. independent replay ($H_N = T$)	20,000	1,120	0
Hybrid semantics vs. exhaustive replay (all slices ≤ 3)	20,000	1,120	0
Lemma 4 at singletons ($F(\emptyset) \subseteq F(\{v\})$)	20,000	1,120	0
Static keys persist in every slice (Thm. 4c)	20,000	1,120	0
Confirmed findings vs. run-enumeration oracle (≤ 2)	20,000	1,120	0
Confirmed findings are symptoms (Thm. 3b)	20,000	1,120	0
Confirmed findings are monotone (Thm. 5c)	20,000	1,120	0
Whole slice is exact (Thm. 3a)	20,000	1,120	0
Partition summaries compose (Thm. 2d)	20,000	1,120	0
Incremental = full re-verification, incl. edges (Thm. 8)	20,000	1,120	0
Run semantics vs. all evidence models (Lemmas W, M)	8,000	0	0

after node and edge changes. The model oracle agrees with the run semantics on 8,000 teams. The reductions agree with a SAT solver on all formulas (0 disagreements for goal keys, 0 for robust keys, which are present exactly when the formula is unsatisfiable), and the view-disjunctive constructions behave as Theorem 4b states (all conditions hold). These checks cover small teams only; they are evidence, not proof, for larger teams.

Conformance testing found two real bugs, both logged in the replication package. A switch with one successor was treated as choosing nothing; every evaluator read the same field the same way, so the oracles agreed with the checker until a theorem check exposed it. And the first version of lifting fixed the witness's contributor sets, which missed liftable witnesses that differ only in which entry edge or which non-minimal contributor set they use; the run oracle exposed it. Oracles that share a data model can share a misreading of it; we return to this in Section 4.7.

Table 7. Mutation Benchmark: Detection (% of All and of Hidden Keys), False Findings, and Exact Localization of Hidden Keys

Checker	All	Hidden	False/reported	Exact loc.
B1 agent (conf.)	63.9	0.0	0/2720	–
B1 agent (raw)	63.9	0.1	201/2834	0.0
SL (conf.)	99.9	99.9	0/4179	86.3
SL (raw)	99.9	99.9	283/4332	86.0
B6 must	99.9	100.0	129/3988	–
B5 smells	27.0	62.1	2107/3191	0.0
B4 lint	31.0	0.0	59/1463	–
B2 + slice shrinking	100.0	100.0	–	99.2
B2 deviation set	100.0	100.0	–	1.0

Answer to RQ1

Yes, with qualifications. A slice is a convex set of agents with a hybrid semantics, per-exit derived contracts, and emergent obligations defined over witnesses. Raw slice findings approximate in both directions; confirmed findings are sound (Theorem 3b) and monotone in the slice (Theorem 5c). The singleton slicing is agent-level analysis; larger slices recover every confirmed agent-level finding and reveal defect families that it misses (Theorem 4a,c), so confirmed slice-level analysis is strictly more expressive. Analyses that judge one agent's view at a time cannot separate safe from unsafe teams on S2, S4 and S6a (Theorem 4b); analyses that combine views, such as a global must-analysis, are not covered. Slice abstraction is compositional over partitions with an acyclic quotient and a global failure set (Theorem 2d); exact slice contracts are not single set-valued assumption-guarantee pairs (Theorem 2c). Confirmation needs a whole-team search, so confirmed analysis is not local (Theorem 7b). Per-node steps are SMT-checked; the inductions are proved on paper, some steps in sketch form; 219,200 oracle comparisons on small teams show no mismatch.

4.4 RQ2: Detection and Localization Power

4.4.1 Defects from engineering changes (E8). Of the 1,680 mutated teams, 1,345 have at least one symptom and 1,220 are unsafe; together they have 3,863 symptom keys. The replay oracle agrees with the exact semantics on all 840 teams it checked (0 disagreements).

How often are defects hidden? Most symptoms are visible to an agent-by-agent check: 2,467 keys have depth 1. But 1,396 keys (36.1%) have depth at least two: 951 of depth 2, 142 of depth 3, 82 of depth 4, 59 of depth 5 and 10 of depth 6. 152 of the hidden keys are deeper than the enumeration bound. 502 teams have at least one hidden key, and in 210 of the 1,345 defective teams (190 of them unsafe) the confirmed agent-level check reports nothing, so it would declare the team clean. The hidden share depends on team size: 28.3% at 8 nodes, 37.3% at 12, 40.8% at 22 and 37.3% at 32 (Table 9). Table 8 shows which changes produce hidden defects: relaxing a join is the most dangerous change, followed by letting two agents share a model.

Detection and false findings. Table 7 compares the checkers. Confirmed agent-level analysis (B1) finds 63.9% of all keys and 0.0% of the hidden ones. Confirmed SL finds 99.9% of all keys and 99.9% of the hidden ones with no false finding (0 false keys); the few it misses are deeper than any of its slices. Without confirmation, SL would report 283 false keys (6.5% of its findings), and

Table 8. Mutation Benchmark by Change Operator: Unclean Teams Containing the Operator, Their Keys, and the Share of Keys That Are Hidden

Operator	Teams	Keys	Hidden (%)
add requirement	501	1943	31.9
drop guarantee	445	1501	24.2
fork to switch	138	377	24.7
gate to agent	415	1069	34.2
relax join	369	1318	67.1
remove edge	94	230	22.6
share oracle	461	1071	36.3

Table 9. Mutation Benchmark by Team Size: Unclean Teams, Keys, Share of Hidden Keys, B1 Recall on Hidden Keys, Exact Localization of SL [95% Cluster-Bootstrap CI] and of Shrinking, and B6 False Rate (%)

Inner	Nodes	Teams	Keys	Hidden	B1	SL exact	Shrink	B6 false
6	8–8	353	928	28.3	0.0	96.6 [92.7, 99.3]	100.0	1.4
10	12–12	331	875	37.3	0.0	88.8 [83.5, 93.6]	100.0	4.3
20	22–22	323	1107	40.8	0.0	83.4 [76.8, 89.2]	98.9	2.9
30	32–32	338	953	37.3	0.0	77.7 [70.8, 83.6]	97.9	4.4

agent-level analysis 7.1%. The global must-analysis B6 is a strong detector: it finds 99.9% of all keys and 100.0% of the hidden ones in linear time per failure set, but 129 of its 3,988 findings (3.2%) are false, because a lower bound on content does not know which nodes are reachable together. The smell detector B5 flags 62.1% of the hidden keys but 66.0% of its findings are false, and the lint B4 finds no hidden key.

Localization. For hidden keys, B6 names only the symptom node, which is never a minimal confirming slice of a hidden key. SL's top slice is an exact minimal slice in 86.3% of cases, falling from 96.6% at 8 nodes to 77.7% at 32 nodes: on larger teams its fixed strategies more often return a confirming slice that is not minimal. Shrinking a whole-team witness slice by slice is exact in 99.2% of cases and minimum-size in 93.3%, at about 29 slice checks per key (median 2.7 ms), and it stays above 97.9% at every size. Combined with the must-analysis as detector, the pipeline detect–confirm–shrink is exact for 99.2% of the hidden keys it detects. Median times per team are 0.1 ms for B6, 0.1 ms for the exact whole-team check, and 2.0 ms for confirmed SL. The deviation set of the whole-team witness, which needs no slices, is exact in only 1.0% of cases and covers 0.36 of the team on average: it names every agent that deviated in the run, including those whose deviation does not matter for the symptom.

4.4.2 Injected defects by class (E2). Table 10 gives recall of the primary key by class on the 4,480 injected teams. The six agent-level classes have depth one and are found by B1. Every injected S2, S3, S4, S6a and S6b defect has depth at least two; B1 finds none of their primary keys (0 of 1,400), and SL finds all of them (1,400). This contrast is a consequence of Theorem 4a, which the gadgets instantiate, not an independent finding; the experiment confirms that theory and implementation agree across all seven topologies. Under the pre-registered metric (any ground-truth key), B1 detects 20.0% of the hidden-class teams: it reports the unmet root assumption of every S6b team (100.0%),

Table 10. Injected Benchmark: Recall (%) of the Primary Injected Symptom by Class and Checker (Small and Medium). Depth From Exhaustive Confirmed Enumeration (Small)

Class	Depth	B1	SL	Win-2	Band-1	Region	Goal	B2	B3	B4	B5	B6
A1	1	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	0.0	100.0
A2	1	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	0.0	100.0
A3	1	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	0.0	0.0	100.0
S1v	1	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	0.0	100.0
A4	1	100.0	100.0	100.0	100.0	0.0	100.0	100.0	100.0	100.0	0.0	100.0
S5	1	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	0.0	100.0	100.0
S2	2	0.0	100.0	100.0	0.0	100.0	100.0	100.0	99.6	0.0	100.0	100.0
S3	2	0.0	100.0	100.0	0.0	100.0	100.0	100.0	100.0	0.0	100.0	100.0
S4	2–4	0.0	100.0	34.6	0.0	0.0	100.0	100.0	98.2	0.0	100.0	100.0
S6a	2–4	0.0	100.0	28.2	0.0	0.0	100.0	100.0	96.1	0.0	100.0	100.0
S6b	2–4	0.0	100.0	34.6	0.0	0.0	100.0	100.0	98.9	0.0	0.0	100.0

Win-2: window-2; Goal: goal-directed.

Table 11. Localization of Hidden Injected Defects (Small Benchmark, Detected Cases). Exact: Top Slice Is a True Minimal Confirming Slice [95% Cluster-Bootstrap CI]; Minimum: Its Size Equals the Depth; Precise: Within the Root-Cause Hull; Hit: Meets the Root Cause; Cost: Slice Checks

Checker	n	Exact [95% CI]	Min.	Precise	Hit	Size	Ratio	Cost
SL (combined)	700	86.9 [85.3, 88.4]	86.9	86.9	100.0	2.8	0.21	36
goal-directed	700	40.0 [40.0, 40.0]	40.0	40.0	100.0	3.4	0.25	8
region	280	0.0 [0.0, 0.0]	0.0	100.0	100.0	3.2	0.23	5
window-2	416	100.0 [100.0, 100.0]	100.0	100.0	100.0	2.0	0.15	13
B3 random slices	687	49.8 [46.3, 53.3]	49.8	53.6	100.0	4.5	0.34	36
B2 witness lineage hull	700	0.0 [0.0, 0.0]	0.0	0.0	100.0	12.4	0.92	1
B2 witness deviation set	700	0.0 [0.0, 0.0]	38.9	0.0	100.0	3.4	0.26	1
B2 + slice shrinking	700	100.0 [100.0, 100.0]	90.0	100.0	100.0	2.7	0.20	15

but never the goal that is lost. The must-analysis B6 also finds every primary key here, and raises no alarm on clean or decoy teams.

Strategies matter. Band slices cut across parallel branches but never contain a join together with a branch, nor two components at different depths, so band-1 misses every hidden class. Region slices find S2 and S3 but miss S4 and S6, whose interacting components are not inside one single-entry single-exit region. Window-2 misses S4 and S6 when more than one hand-off separates the interacting components. Goal-directed slices find every class. Random slices of matched size find 98.6%: on teams of this size, many slices happen to contain the interacting pair.

Table 12. Exact Localization (%) of Hidden Injected Defects by Class (Small Benchmark)

Class	SL	B3 random	B2 deviation	B2+shrink
S2	100.0	67.9	0.0	100.0
S3	100.0	80.0	0.0	100.0
S4	34.3	29.9	0.0	100.0
S6a	100.0	33.3	0.0	100.0
S6b	100.0	36.2	0.0	100.0

Table 13. False Alarms on 280 Clean and 1,120 Decoy Teams: Teams With Any Reported Key (%)

Checker	Clean	Decoy
B1 agent (conf.)	0/280 (0.0)	0/1120 (0.0)
SL (conf.)	0/280 (0.0)	0/1120 (0.0)
window-2	0/280 (0.0)	0/1120 (0.0)
band-1	0/280 (0.0)	0/1120 (0.0)
region	0/280 (0.0)	0/1120 (0.0)
goal-dir.	0/280 (0.0)	0/1120 (0.0)
B2 whole	0/280 (0.0)	0/1120 (0.0)
B3 random	0/280 (0.0)	0/1120 (0.0)
B4 lint	0/280 (0.0)	0/1120 (0.0)
B5 smells	17/280 (6.1)	857/1120 (76.5)
B6 must	0/280 (0.0)	0/1120 (0.0)
B1 agent (raw)	0/280 (0.0)	0/1120 (0.0)
SL (raw)	0/280 (0.0)	0/1120 (0.0)

Localization. Table 11 reports localization on the small benchmark, and Table 12 breaks it down by class. The witness lineage hull covers 0.92 of the team and is never minimal; as noted above, this is a property of the lineage hull, not an empirical finding, and we keep it only for reference. The witness deviation set is a fair whole-team baseline: it is small (0.26 of the team) but exact in 0.0% of cases, because it always includes nodes that deviated without mattering (all agents on a failed shared model, for instance). SL is exact in 86.9% [85.3%, 88.4%] of cases, against 49.8% for random slices of matched size (Holm-adjusted Wilcoxon $p < 0.001$, $\hat{A}_{12} = 0.99$), and better than the deviation set ($p < 0.001$, $\hat{A}_{12} = 1.00$). The aggregate hides one weak class: on S4, SL's top slice is exact in only 34.3% of cases, close to random slices: no SL strategy targets the pair of claimers of one critical goal, so its top slice is usually a larger confirming slice than the minimal hull of the pair. Shrinking a whole-team witness is exact in 100.0% of cases (minimum-size in 90.0%) at about 15 slice checks, and it is better than SL ($\hat{A}_{12} = 0.96$). On the medium benchmark, without exhaustive ground truth, SL's top slice covers 0.088 of the team and is precise in 87.0% of cases; shrinking covers 0.087.

False alarms. On clean and decoy teams, every checker except the smell detector is silent (Table 13); the one-sided 95% upper bounds on the alarm rate of SL are 1.1% (clean) and 0.3% (decoys). B5 alarms on 76.5% of decoys: the shape of a defect is not the defect. Because the gadgets do not interact with their hosts, these numbers say little about unstructured teams; E8 and E7 do.

Table 14. Encoded Published Designs (Documented Configuration): Most Informative Finding, Class, Depth, Minimal Confirming Slice, and Whether Agent-Level Analysis (AL), the Smell Detector (B5) and the Must-Analysis (B6) Report the Key

Design	$\|N\|$	Key	Class	Depth	Minimal slice	AL	B5	B6
[26] MetaGPT	8	robust: tests_pass	S6a	2	QA, TestRun	no	yes	yes
[50] ChatDev	8	robust: correct	A4	1	Testing	yes	no	yes
[29] AgentCoder	5	robust: tests_pass	S6a	2	Programmer, Executor	no	yes	yes
[31] MapCoder	7	robust: samples_pass	S6a	2	Coding, SampleIO	no	yes	yes
[17] SelfCollab	5	robust: tested	A4	1	Tester	yes	no	yes
[18] Debate	9	robust: answer_ok	S4	3	R1, R2, R3	no	yes	yes
[59] MoA	9	robust: answer_ok	A4	1	Aggregator	yes	no	yes
[63] AutoGenGroup	7	goal: runs	S6b	3	Coder, Merge, Executor	no	no	yes
[64] Agentless	9	robust: issue_fixed	S6a	3	ReproGen, Validate1, Validate2	no	yes	yes

4.4.3 Published designs (E3). We encoded nine published designs with the protocol below and analyzed them with confirmed exhaustive slices up to size 5.

- P1 Each documented role or pipeline stage is a node. A step described as a program (test execution, compilation, a sample input/output check) is a gate. Every LLM role is an agent whose oracle is the backbone model of the paper's main configuration.
- P2 Edges follow the documented hand-off order. Parallel roles fork; aggregation steps are joins with the documented policy (concatenate or synthesize: all; majority: quorum; first or selected candidate: any). A manager that picks the next speaker is a switch.
- P3 A role guarantees the artifact it produces and requires every artifact it consumes. Goals are the quality claims the design's evaluation relies on; the claim resting on executed tests is critical, with $f = 1$.
- P4 Iterative loops (debate rounds, debugging loops, group-chat turns) are unrolled once.

All contracts were declared by one team of encoders (us) from the published descriptions; there is no second encoder. They are hypotheses about the designs, not measurements of their behavior. P3 with $f = 1$ asks a specific question of every design: does any critical claim rest on a single model? Table 14 lists the most informative finding per design, and Table 15 varies the oracle assignment, the fault budget and the join policy.

Under the documented configuration, all 9 designs have a symptom, and 6 have one that agent-level analysis does not report. With $f = 0$, 8 of the nine are safe: the findings are about robustness to one model failure, which P3 deliberately asks about. The recurring pattern is S6a: a deterministic gate (MetaGPT's test run, AgentCoder's executor, MapCoder's sample input/output check, Agentless's validation runs) certifies a critical property using tests or code written by an LLM agent, so the gate's guarantee is only as robust as that agent's model. The minimal slice {writer, gate} says exactly that. The repair is to back the gate's input with an independent source, as AgentCoder does for its tests but not for its code; in the running example, an independent test designer on another model makes the chain safe (0 symptoms). In multi-agent debate, the three revisers run on one backbone (S4, depth 3); with distinct models the finding changes to S6a (the vote rests on the round-one answers). ChatDev's critical claim rests on one LLM tester (A4, visible at agent level). The AutoGen

Table 15. Sensitivity of E3 Findings to the Encoding: Documented Oracles, One Shared Oracle, Distinct Oracles per Agent, No Faults ($f = 0$), and an Alternative Join Policy

Design	doc.	shared	distinct	$f = 0$	alt. join
MetaGPT	S6a	S6a	S6a	safe	–
ChatDev	A4	A4	A4	safe	–
AgentCoder	S6a	S6a	S6a	safe	S3/S6b
MapCoder	S6a	S6a	S6a	safe	–
SelfCollab	A4	A4	A4	safe	–
Debate	S4	S4	S6a	safe	S2
MoA	A4	A4	A4	safe	S3/S6b
AutoGenGroup	S5/S6b	S5/S6b	S5/S6b	S5/S6b	–
Agentless	S6a	S6a	S6a	safe	S6a

group-chat encoding loses runs when the manager routes to the critic (S6b, depth 3, together with depth-one bypass findings); this finding depends on unrolling the chat once. The smell detector reports the hidden finding in 5 and the must-analysis in 6 of the 6 designs, but neither says which agents form the defect. These are findings about our encodings. In particular, a robust symptom says only that the critical claim is not established when one model fails; it does not distinguish a gate that would then falsely certify (weak LLM-written tests that pass) from one that would visibly fail (MapCoder's sample check uses input/output pairs from the problem statement, so a wrong program fails it). Whether the real systems mitigate the findings, for example by retries, repeated sampling or human review, is outside our model.

Answer to RQ2

On defects induced by random engineering changes, 36.1% of the symptoms have depth at least two, a share that is higher on larger teams; an agent-by-agent check reports none of them by definition, and for 210 of 1,345 defective teams it reports nothing at all. Confirmed slice analysis finds 99.9% of the hidden symptoms with no false finding; raw slice findings are 6.5% false. A global must-analysis detects 100.0% of the hidden symptoms but 3.2% of its findings are false and it names only the symptom node. For localization, shrinking a whole-team witness slice by slice is exact for 99.2% of hidden symptoms, the combined slicing strategy for 86.3% (less on larger teams), and the witness's deviation set for 1.0%. In 6 of nine encoded designs, our declared contracts admit a defect that agent-level analysis does not report (the must-analysis reports it); mostly, a gate's critical claim rests on an artifact written by one model.

4.5 RQ3: Cost, Slicing Strategy, and Evolution

Slice size and hand-off distance (E4). How large must slices be? On the injected benchmark, recall of exhaustive-k rises from 54.5% at $k = 1$ to 81.6% at $k = 2$, 90.3% at $k = 3$ and 100.0% at $k = 4$; but this curve only restates the injector, which draws the number of hand-offs ℓ between interacting components from $\{0, 1, 2\}$. We therefore ran a dedicated sweep (E4b, 1,200 teams) with $\ell = 0, \ldots, 5$ and confirmed exhaustive slices up to $k = 8$. Fig. 3 shows the result: for S4, S6a and S6b the depth is exactly $\ell + 2$ (in 100.0% of teams), and for S2 and S3 it is always 2 (100.0%), whatever the length of the branches. This is what Theorem 4a predicts from the gadget shape and convexity, so

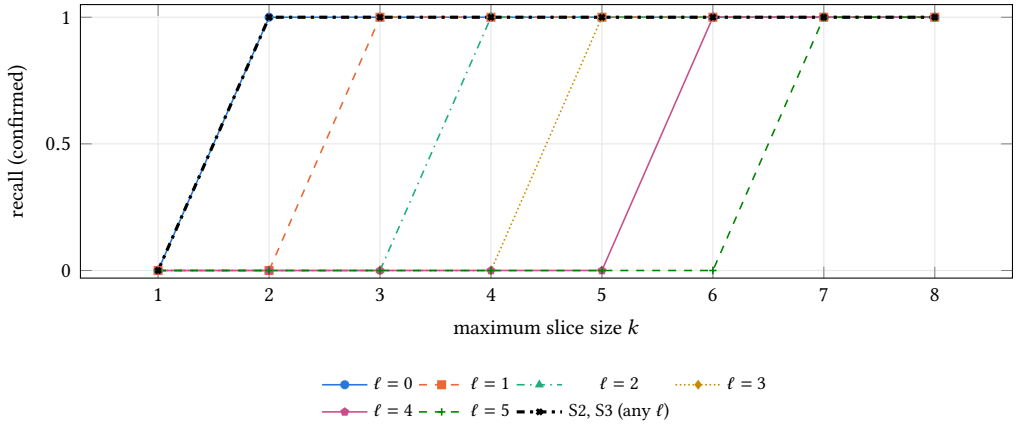


Fig. 3. Recall (confirmed) of the primary injected symptom by maximum slice size k, for hand-off distances $\ell = 0, \ldots, 5$ (S4, S6a, S6b pooled) and for join defects (S2, S3). The depth needed is $\ell + 2$ for chains of conditional or correlated claims and 2 for joins.

E4b confirms the implementation rather than discovering a law. To check that depth does not grow with the rest of the team, we repeated the sweep with a 12-node host (17–33 nodes, enumerating slices up to size $\ell + 2$): the depth is again $\ell + 2$ (100.0%) and 2 for joins (100.0%). The number of slices grows with k (32 slices per team at $k = 2$, 155 at $k = 8$), and no clean or decoy team raised an alarm at any k (0.0%). On the mutation benchmark, 2,467 of 3,863 symptoms have depth one and only 59 and 10 have depth five and six.

Scalability (E5). Published agent teams have tens of agents, a range that E8 covers with confirmed analysis; E5 asks how the methods behave far beyond it, to see where each one breaks. We generate teams of 10 to 5,000 nodes in four topologies, inject one hidden defect, and run each strategy with a budget of 60 s per team (360 teams). E5 measures raw slice analysis; confirmation adds one lift search per finding. Time-outs are censored at the budget, and a defect counts as detected only when the run finished. Fig. 4 and Table 16 report wall time. On the three switch-free topologies every strategy finishes on almost every team and the censored log-log slopes stay close to 2 (SL: 1.94, upper bound 2.03; window-2: 1.70; band-1: 1.26). But on these teams the exact whole-team checker is the cheapest method of all (slope 1.14; 0.02 s at 5,000 nodes): without switches and with all-joins a team has one fault-free run plus one run per failure set, and checking it is a linear pass. The global must-analysis is also cheap (1.18 s at 5,000 nodes) and detects every defect. On the *mixed* topology the picture changes. The whole-team checker's cost grows with the number of choice points and it exceeds the budget on 36 of 90 teams; singleton and window-2 slices time out on 0 and 0 of them; but SL times out on 30 teams, because its goal-directed and region slices are hulls that grow with the team and contain many choice points (239 slices exceeded the per-slice run budget and were skipped). Counted over all teams, SL detects the defect in 60 of 90 mixed teams and the whole-team checker in 54. Under the pre-registered decision rule, H3.1 is *not supported*: the polynomial strategies exceed slope 2 on mixed teams, and whole-team checking is not super-polynomial on switch-free teams. The refined reading, that cost is driven by the choice points inside a slice rather than by the number of members, is exploratory. Peak memory of SL stayed below 54 MB. Confirmation adds cost on top of raw slicing. On teams of up to 200 nodes we also ran confirmed SL: at 200 nodes its median time is 0.06 s on switch-free teams and 0.68 s on mixed teams (raw SL: 0.06 s and 0.80 s), it times out on

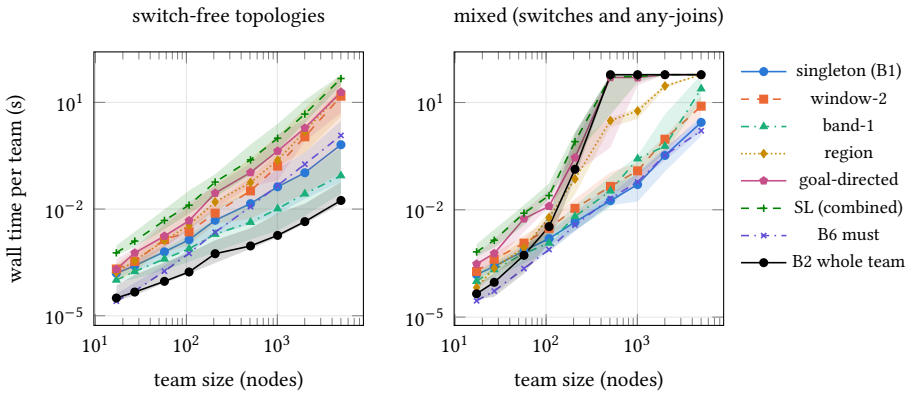


Fig. 4. Wall time per team versus team size (log-log), median and interquartile band over seeds and topologies, with time-outs counted at the budget. Left: switch-free topologies (layered, series-parallel, random). Right: mixed topologies with switches and any-joins.

Table 16. Scalability: Log-Log Slope of Time vs. Size ($|N| \geq 50$) for Finished Runs Only and With Time-Outs Censored at the Budget [95% Bootstrap CI], Median Time at 5,000 Nodes (Censored), Time-Outs, and Defects Detected by Finished Runs

Strategy	Topology	Slope (finished)	Slope (censored)	s @5k	Time-outs	Detected
singleton (B1)	free	1.51 [1.38, 1.64]	1.51 [1.37, 1.64]	0.64	0/270	0/270
singleton (B1)	mixed	1.73 [1.56, 1.89]	1.73 [1.55, 1.90]	2.78	0/90	0/90
window-2	free	1.70 [1.54, 1.87]	1.70 [1.53, 1.87]	14.90	0/270	198/270
window-2	mixed	1.88 [1.71, 2.04]	1.88 [1.70, 2.05]	7.92	0/90	64/90
band-1	free	1.26 [1.14, 1.37]	1.26 [1.15, 1.38]	0.09	0/270	0/270
band-1	mixed	2.20 [2.01, 2.38]	2.33 [2.15, 2.52]	24.18	4/90	0/90
region	free	1.76 [1.59, 1.92]	1.88 [1.72, 2.05]	19.44	9/270	161/270
region	mixed	2.93 [2.74, 3.15]	2.59 [2.40, 2.78]	60.00	12/90	46/90
goal-directed	free	1.82 [1.70, 1.94]	1.91 [1.79, 2.03]	19.30	8/270	262/270
goal-directed	mixed	2.76 [2.35, 3.17]	2.34 [2.13, 2.60]	60.00	29/90	61/90
SL (combined)	free	1.87 [1.77, 1.98]	1.94 [1.85, 2.03]	46.82	13/270	257/270
SL (combined)	mixed	3.03 [2.74, 3.38]	2.20 [1.97, 2.46]	60.00	30/90	60/90
B6 must	free	1.94 [1.92, 1.96]	1.94 [1.92, 1.96]	1.18	0/270	270/270
B6 must	mixed	2.02 [1.95, 2.10]	2.02 [1.95, 2.10]	1.62	0/90	90/90
B2 whole team	free	1.14 [1.07, 1.21]	1.14 [1.07, 1.21]	0.02	0/270	270/270
B2 whole team	mixed	4.38 [3.39, 5.54]	2.89 [2.55, 3.24]	60.00	36/90	54/90

0 of 150 switch-free and 0 of 50 mixed teams of at most 200 nodes, and 0 lift searches exceeded their budget.

Evolution (E6). We take 1,967 random changes on 210 teams of 20, 40 and 80 nodes (seven topologies, ten seeds; teams whose full verification exceeds 60 s are skipped, which happened 0 times): a change of a requirement, guarantee or declared evidence, of an agent's kind or oracle, of a routing or join policy, or the addition or removal of an edge. After each change we re-verify incrementally with Theorem 8 and compare with full re-verification of all SL slices. The incremental

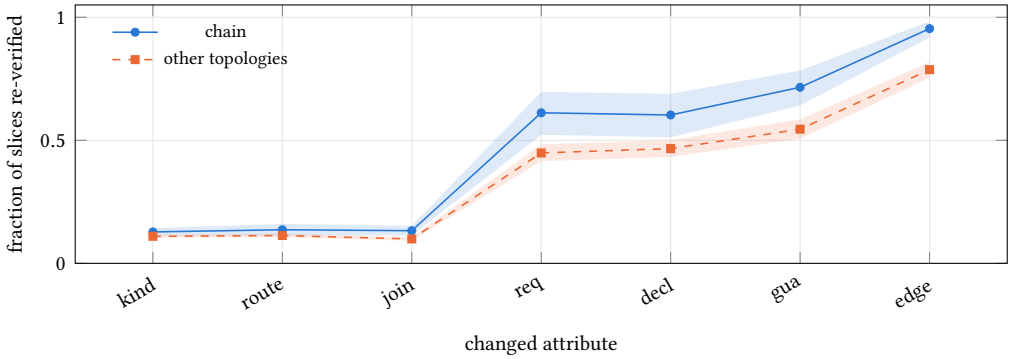


Fig. 5. Fraction of SL slices that incremental recertification re-verifies, by changed attribute, for chains and for other topologies; bands are 95% bootstrap CIs.

Table 17. Recertification: Share (%) of SL Slices Re-Verified per Change Kind and Topology

Change	*n*	chain	tree	f.-j.	lay.	s.-p.	rand.	mixed	all
kind	465	12.8	8.1	15.1	13.5	10.0	10.7	9.8	11.3
route	258	13.6	9.4	14.6	13.9	10.3	9.8	9.4	11.6
join	226	13.3	7.9	11.9	11.0	9.6	8.2	10.4	10.4
req	248	61.2	36.8	58.9	52.0	37.9	31.7	54.9	46.8
decl	251	60.3	29.4	64.3	57.6	43.0	40.6	40.9	48.5
gua	229	71.5	36.3	72.6	61.1	60.7	37.3	53.2	57.3
edge	290	95.4	62.5	95.9	88.7	70.6	69.2	80.4	80.7

result agrees with full re-verification in 100.0% of the changes. On non-chain topologies 35.0% of the slices are re-verified (95% CI upper bound 36.6%), against 42.0% on chains; the share falls with team size (41.5%, 34.5% and 32.1% for 20, 40 and 80 nodes). Fig. 5 and Table 17 show that the saving depends on the change: changes of kind, routing or join policy re-verify only slices containing the node (11.3%, 11.6%, 10.4%), contract changes about half (46.8% for requirements, 57.3% for guarantees), and edge changes 80.7%, because they change ancestor relations. In wall time the saving is small: the median speed-up is 1.9×, and in 17.6% of the changes incremental re-verification is *slower* than full re-verification, because on teams of this size each slice is checked in microseconds and computing the dependency set is a noticeable share of the cost. The share of slices saved is the robust result; the time saved depends on how expensive slices are.

False findings, sensitivity and ablation (E7). *False findings on unstructured teams.* The injected benchmark is built from clean teams, which may flatter precision. On 3,000 random wild teams with arbitrary contracts, policies and switches, we ran every convex slice of size at most 3: 16,510 of 573,242 raw findings (2.9%) are not symptoms of the team, and 8.6% of the 140,625 slices report at least one false key. These are the false alarms of Theorem 3c. Confirmation removes all of them (0 false among 563,243 confirmed findings) at a total lifting cost of 9.9 s. It also drops 357 findings that are true symptoms of the team but whose witnesses in that slice do not lift; a larger slice confirms them (Theorem 5c). Non-convex slices (a convex slice minus one interior node) have a false-finding rate of 3.2% on the same teams, close to convex ones; in our data convexity is justified by the semantics (a well-defined environment, Lemma 3, and the abstraction theorem), not by fewer false

Table 18. Ablation on Hidden Injected Classes (Teams of 12 Nodes): Recall (%) of SL With the Full Semantics, With Every Join Treated as all, and Without Fault Reasoning; and False Keys of Non-Convex Slices

Class	full	joins as all	no faults	non-convex false keys
S2	100.0	0.0	100.0	0
S3	100.0	0.0	100.0	0
S4	100.0	100.0	0.0	0
S6a	100.0	100.0	0.0	0
S6b	100.0	100.0	100.0	0

findings. *Sensitivity.* Varying the share of gates, the number of distinct oracles and the number of requirements per node on 3,240 injected teams of 15 nodes does not change the per-class outcome; since the gadget is fixed per class, this is expected. *Ablation* (Table 18). If the analysis treats every join as all, S2 and S3 are no longer detected (0.0% and 0.0%); without fault reasoning, S4 and S6a are lost (0.0% and 0.0%). Each part of the semantics is responsible for specific classes.

Answer to RQ3

Slices must be as large as the distance between the interacting agents plus two: depth is $\ell + 2$ for ℓ hand-offs and 2 for joins, independently of the host's size. Small slices (singleton, window-2, band) scale polynomially to 5,011 nodes; on mixed teams singleton and window-2 time out on 0 and 0 of 90 teams and band-1 on 4. Confirmed SL on teams of up to 200 nodes never exceeded the budget (0 time-outs on mixed teams). Large slices pay for the choice points they contain: on choice-rich teams SL times out on 30 of 90 teams and whole-team checking on 36; on switch-free teams whole-team checking and the must-analysis are cheaper than any slicing. The pre-registered scalability hypothesis is therefore not supported. Incremental recertification is always correct and re-verifies 35.0% of the slices on non-chain topologies, but saves little wall time on small slices (median 1.9×). Raw slice findings are 2.9% false on unstructured teams; confirmation removes them all.

4.6 Discussion

What slices add, and what they do not. Detection alone does not need slices: an exact whole-team check finds every symptom, and a linear global must-analysis finds almost all of them. What slices add is *which agents*. A confirmed finding comes with the smallest set of agents whose joint behavior already exhibits it, together with the obligation that fails (joint coverage, non-bypass, conditional backing, independence). Neither a whole-team witness (whose deviation set names every agent that misbehaved, relevant or not) nor a must-analysis (which names the node where a fact is missing) gives that. Confirmation removes the false findings that face value would otherwise produce.

Which defects hide? Every defect whose mechanism is a join policy, a shared model, or a conditional guarantee resting on a fallible provision is hidden at agent level (Proposition 1). Defects whose mechanism is a missing claim are visible. In the mutation benchmark, relaxing an all-join is the change most likely to create a hidden defect. An any or quorum join is not dangerous by itself: the decoy whose branches all claim every goal is safe. The danger is a join whose branches carry *different* guarantees, which slices see because they compute what each branch establishes.

How large should slices be? As large as the distance between the interacting agents, plus two. For joins this is two, whatever the branch length. For conditional and correlated claims it grows by one per hand-off. Larger slices only enlarge explanations.

Choosing a strategy. For certification, use the whole team when it has few switches and quorum joins: it is exact and cheap. For choice-rich teams, a partition with an acyclic quotient keeps each part's choices local (Theorem 2d); we did not measure its cost. For diagnosis, the most precise localization in our experiments came from the pipeline *detect, confirm, shrink*: detect with the whole team or with the must-analysis, confirm each finding against the team, and shrink the slice of each confirmed finding until no member can be removed. The slicing strategies are then useful as candidate generators when a whole-team run is too expensive, as on choice-rich teams: the combined strategy (singleton, window-2, region, goal-directed) needs no whole-team run. Goal-directed slices are the best single strategy for detection; window-2 gives the smallest explanations and timed out on 0 of 270 switch-free and 0 of 90 mixed teams in E5; region slices give the most readable ones. On large choice-rich teams, slices should be capped by their number of choice points rather than their number of members, since choice points drive the cost (Theorem 7b).

Using the findings. A practitioner obtains contracts from the artifacts each role reads and writes (prompts, tool schemas, orchestration code) and from the checks the pipeline runs. A finding then suggests a repair through its obligation: a joint-coverage gap suggests an all or quorum policy, or claims on every branch; correlated evidence suggests distinct models for redundant claimers; a conditional collapse suggests an independent source for the gate's input; a bypass suggests a check after the merge. The running example shows that a plausible repair can fail: a deterministic test generator placed after the developer still depends on the developer's model (depth 4 instead of 3), while an independent test designer on another model makes the chain safe.

Slices versus failure attribution. Automated failure attribution names the agent and step responsible for a failed run [67]. Slices answer a different question before any run: which group of agents, under which obligation, makes a failure possible. A minimal confirming slice is a hypothesis for where attribution should look.

4.7 Threats to Validity

Construct validity. Contracts are declared, not measured. Whether an LLM agent establishes design, or a gate's tests are adequate, is outside the model. Our findings are relative to the declared contracts. Properties are Boolean; real quality is graded. A model failure is modeled as "the claimed property does not hold", which covers wrong artifacts presented as right, but not partial degradation. No experiment links a finding to an observed failure of a real system; mapping slice findings to failure traces [8] or to fault-injection runs of open-source pipelines is future work.

Internal validity. The generator, injector, mutation operators and checkers come from the same authors and share the data model. We mitigated this with three independent oracles (model, replay, run), a fresh exact verification of every ground-truth key, proof mutants, decoys and ablations; two bugs were found this way (Section 4.3), and others may remain. Ground-truth depth and minimal slices use the hybrid semantics; confirmation, which decides them, is checked against the run oracle only on small teams. The injected benchmark's per-class results are close to guaranteed by Theorem 4; its value is to test that theory and implementation agree across topologies. The mutation benchmark avoids choosing classes and depths, but its operator set and its uniform mix are our choice. The revision after the first review round changed the method and analyses (deviation log): the detection metric now reports the pre-registered definition alongside the primary key, ground truth uses slices up to size 6 on small teams as planned, degenerate tests were replaced by counts, and the E3 encodings were revised for consistency with the protocol, which changed some findings in both directions.

External validity. Teams are acyclic; loops are unrolled once, and shared mutable state is not modeled. The nine encoded designs are a sample encoded by one team of authors without a second encoder or agreement measure; other encoders might choose other contracts, which is why we report a sensitivity analysis and describe the findings as properties of our encodings. Teams in the conformance and mutation experiments are small.

Conclusion validity. Many outcomes are deterministic per class; we report counts and exact bounds rather than tests for them. Localization comparisons use ten seeds per topology, with cluster-bootstrap intervals and Holm correction. In E5, time-outs are censored at the budget, so slopes for strategies with many time-outs are lower bounds; E5 also rotates the injected class with the seed, so class and seed are confounded there, and S5 and S6b are not injected.

5 Related Work

LLM multi-agent systems for software engineering. Role-based teams (MetaGPT [26], ChatDev [50], self-collaboration [17]), code-generation pipelines (AgentCoder [29], MapCoder [31]), issue-resolution pipelines (Agentless [64], MAGIS [55]) and general frameworks (AutoGen [63], CAMEL [41], multi-agent debate [18], mixture-of-agents [59]) define the topologies we encode. He et al. survey LLM-based multi-agent systems in software engineering and call for principled ways to engineer them [24], as do broader surveys [20, 28]. None of these works reasons about sub-teams as contracts before execution.

Evaluating, testing and guarding agents. Benchmarks evaluate run outputs [32, 43, 65]; ToolEmu emulates tools to expose risky actions [53]; AgentDojo and indirect prompt injection attack single agents [16, 22]. Failure taxonomies and attribution [8, 67] analyze executions after the fact. Guardrails filter one model's inputs and outputs [30, 51], and AgentSpec enforces rules on one agent's actions at run time [58]. In our model a guardrail is a gate or an agent with a contract; slicing asks whether the guardrails, placed where they are, justify the team's goals.

Contracts and compositional reasoning. Design by contract [46], assume-guarantee reasoning for networks of processes [47], rely-guarantee [34], composition of specifications [1], interface automata [14] and contract theories [5] provide the foundation. Hierarchical contract-based design checks refinement level by level over a component hierarchy, as in OCRA [9] and AGREE [12]. There, the decomposition is given by the architecture; we choose sub-structures by strategy or by shrinking, allow non-connected ones, and measure defects by the size of the smallest sub-structure that exposes them. Circular assume-guarantee rules are sound only under side conditions [45, 48]; our acyclic-quotient condition is a simple instance for dataflow teams. Learning assumptions [11] computes a component's weakest assumption; our derived entry assumptions are the analogue for a slice, and our observation that exact slice contracts are not single set-valued pairs echoes that interfaces need input-dependent guarantees; predicate-based contracts, as in OCRA and AGREE, can state the disjunctive guarantee of Theorem 2c directly.

Abstraction and counterexample explanation. Analyzing some components exactly and abstracting the rest is localization abstraction, refined in CEGAR when an abstract counterexample is spurious [10]. Our confirmation step plays the role of the spuriousness check, and shrinking plays the role of a search for the smallest precise sub-model; unlike CEGAR, we use the size of that sub-model as the result. Counterexample explanation localizes errors in traces by comparing them with correct traces [4, 23]; our witness deviation set is a simple instance, which our experiments show to be imprecise for relational defects. Fault localization ranks program elements [35, 62], and delta debugging isolates failure-inducing changes [66]; shrinking is delta debugging over slices.

Program slicing, regions and convex subgraphs. Weiser's slices [60] and dependence-graph slicing [21, 27] select statements relevant to a criterion; our goal-directed strategy is a backward slice over the team graph, but we prove no projection property. Single-entry single-exit regions and

dominators [33, 39] define our region strategy; workflow nets [56] are a related formalism for fork and join. Enumerating convex subgraphs of DAGs is well studied in instruction-set extension [49], which bounds the same exponential growth we observe.

Diagnosis, redundancy and safety analysis. Model-based diagnosis computes minimal sets of components whose abnormality explains an observation [52]; minimal confirming slices are minimal sets of components whose *exact* analysis exposes a symptom, a dual notion computed before any observation. Fault trees and their minimal cut sets [54], and contract-based fault-tree generation [6], analyze how component failures combine; a minimal confirming slice of a robust key is close to a minimal cut set, S4 is a common-cause failure, and S6a a dependent failure. N-version programming relies on independent versions [3]; Knight and Leveson showed that independently developed versions fail together more often than independence predicts [38], and Littlewood and Miller modeled coincident failures [42]. Feature interactions [7] are an earlier case of defects that arise only in combination, and systems-theoretic safety analysis treats safety as a property of interactions [40]. Model checkers for multi-agent systems verify epistemic and temporal properties [44]; we target a simpler contract-level question.

6 Conclusion and Future Work

A team of AI agents can fail even when every agent keeps its contract, and the failure lives in a relationship among a few agents. Agent slicing makes such relationships analyzable: a slice is a convex set of agents analyzed exactly while the rest is taken at face value, and a finding counts only when its witness lifts to a run of the team. The theory gives sound, monotone confirmed findings, compositional abstraction over acyclic partitions with a global failure set (with declared contracts, only without failures), a separation from analyses that judge one agent's view at a time, and local recertification of raw findings. On defects induced by random engineering changes, 36.1% of the symptoms are not reported by an agent-by-agent check, a share that is higher on larger teams in our data. Confirmed slices find them without false findings. A global must-analysis detects them cheaply but imprecisely; a whole-team witness alone names too many agents; shrinking that witness slice by slice names the few agents involved. Confirmation and shrinking need whole-team searches, so the method is a diagnosis aid, not a local check.

Future work includes validating findings against executions and failure traces of real agent pipelines, extracting team graphs and contracts mechanically from orchestration code, independent encodings, distinguishing a gate that rejects from one that falsely certifies, cycles and retries beyond bounded unrolling, shared mutable state, graded and probabilistic contracts, strategies bounded by choice points, and contract discovery from orchestration code.

A Proofs

Throughout, contents, outputs and keys refer to Definitions 2–7; inductions run over a fixed topological order. We use these per-node facts, each an SMT obligation of Table 5. (L1) If $c \subseteq c'$ and the oracle is sound in the first setting only if it is sound in the second, then $o \subseteq o'$. (L2) The exact output is below the face-value output $c' \cup \mathit{gua}(v)$ whenever $c \subseteq c'$. (L3) The union of the outputs of a contributor subset is below the union of all delivered outputs when each output is pointwise smaller. (L4) Missing requirements $\mathit{req} \setminus (c \cup \mathit{decl})$ are antitone in c. (L5) The output is monotone in the evidence. (L7) Any nonempty selection of entries that all carry env yields env. (L10, L11) A failed oracle yields $o = c$, below the fault-free output. (L12, L13) Declared evidence enables application but never reaches the output.

Lemma 1 (worst case). ($\Leftarrow$) A run is the model with $E(v) = c(v) \cup \mathit{decl}(v)$, and its keys coincide. ($\Rightarrow$) Take a model (ρ, E) and the run ρ itself. Reachability depends only on W and C, which are shared. By induction, the run's output at each node is below the model's (L5 at the node, L3 for

contents), so the run's content is below the model's content and the run's missing requirements, goals and critical goals are supersets of the model's (L4). Goal and assume keys of the model are therefore keys of the run; the robust condition refers to $Sym(T)$ in both cases, and the run misses every critical property the model misses. □

Lemma 2 (minimal choices). Shrinking a contributor set shrinks the content at that node (L3) and, by induction with L1, everywhere below; adding failures shrinks outputs (L10). Reachability does not depend on contributor sets or failures. So every key of a run is a key of the run with the same switch choices, contributor sets of size exactly q chosen inside the original ones, and a failure set of size $\min(f, |Orc|)$ containing the original one. □

Lemma 3 (convexity). If $x \notin \sigma$ were a descendant of $u \in \sigma$ and an ancestor of $w \in \sigma$, the path $u \rightsquigarrow x \rightsquigarrow w$ would leave σ. An outside predecessor x of a member w is an ancestor of w, hence not a descendant of a member. A cycle in the contracted graph must leave and re-enter the contracted node, which gives a path from a member through outside nodes to a member, and conversely. □

Lemma 4 (face value dominates content). By induction we show, for every node v: if v is reached in ρ' then it is reached in ρ, $c_{\rho'}(v) \subseteq c_\rho(v)$ and $o_{\rho'}(v) \subseteq o_\rho(v)$. *Delivery:* if (u, v) is delivered in ρ', then u is reached in ρ; if $u \in \sigma$ the switch choice is the same, and if $u \notin \sigma$ it delivers to all successors in H_σ. *Reachability:* for $v \in \sigma$, entry edges are always delivered in H_σ and member edges are delivered in ρ whenever in ρ', so v's policy is met in ρ; for $v \notin \sigma$, reachability in H_σ needs one delivered edge. *Content:* for $v \in \sigma$ with contributor set C in ρ', the same C is admissible in ρ. Its member contributors have larger outputs in ρ by induction. Its entry contributors carry $env_\sigma(v)$, the union of the face-value outputs of all outside predecessors; each such output dominates the predecessor's output in ρ' (by induction applied with $\emptyset \subseteq \sigma'$, the face-value output is the largest). L3 and L7 give the claim. For $v \notin \sigma$, all delivered in-edges contribute in ρ, a superset of ρ''s contributors. *Output:* for $v \in \sigma$, $orc(v) \in F'$ iff $orc(v) \in F' \cap orc(\sigma)$, so L1 applies; for $v \notin \sigma$, L2 applies. □

Proposition 1. By Lemma 4 with $\emptyset \subseteq \{v\}$, contents in $H_{\{v\}}$ are below face-value contents. Every node is reached in some run of $H_{\{v\}}$: v is reached (all its in-edges are entries) and every other node is reached when v's switch, if any, picks a successor on a path to it; so every static key persists (L4). All of v's in-edges are entries, so its content is $env_{\{v\}}(v) = cone(v)$ for every contributor choice (L7): keys at v are static keys. If v applies and is not a switch, its output is the face-value output and the run equals the face-value run, which has only static keys. □

Theorem 2. (a) Let T' be T with σ contracted to a macro node M with the same entry and exit edges; T' is a DAG by Lemma 3. Fix a global failure set F. A run of T restricts to an entry pattern of σ (delivered entry edges and their contents), an internal run of σ with failures $F \cap orc(\sigma)$, and outside choices. By definition of Beh_σ, the exit outcome and the member keys of the internal run are an outcome of Beh_σ for that pattern. Conversely, an outcome of Beh_σ for the pattern produced by a run of T' comes from an internal run of σ; combined with the outside choices it is a run of T, with the same outside contents, because outside contents depend on σ only through the exits. Content is tracked symbolically as the set of entry edges whose content reaches a member (its flow) plus the properties members add, so each member's content is a function of the entry contents. Hence fault-free runs of T and T' produce the same assume and goal keys (outside keys from outside contents, member keys from the macro), and runs with failures produce the same missing properties at ω; so the robust keys, which are defined from these, coincide. (b) Under the delivery condition, every entry pattern of a run of T delivers all entries or none. If all are delivered and carry at least A^d, validity gives that every member requirement holds and every delivered exit carries at least $A^d \cup G^d$; with smaller entry contents, the replacement component may behave arbitrarily. So every outcome of Beh_σ is an outcome of the replacement, up to larger contents on exits, which can

only remove keys downstream (L1, L4). If T had a goal key, the run exhibiting it would map to a run of the abstracted team with the same choices and no larger contents at ω, which is a goal key there. (c) Obligation C2 shows, over an uninterpreted sort with distinct r_1, r_2, g, that no sets A, G make the exit guarantee $X \cup (\{g\}$ if $r_1 \in X$ or $r_2 \in X)$ equal to $X \cup G$ for $A \subseteq X$ and to X otherwise on the three environments $\emptyset, \{r_1\}, \{r_2\}$: equality at $\{r_1\}$ and $\{r_2\}$ forces $A \subseteq \{r_1\} \cap \{r_2\} = \emptyset$ and $g \in G$, and then equality at $\emptyset$ fails. (d) Induction over the quotient's topological order, applying (a) to one part at a time, with the same global failure set for all parts; each part's entries come from parts already evaluated. Robust keys are decided after composition, from the missing properties at ω and the presence of goal keys. The cyclic example is in Section 3. Each flat contract is valid in isolation, and each part's assumption is the other's guarantee, but in T neither a nor c ever receives what it requires, and ω misses both goals. □

Theorem 3. (a) H_N has no outside node, so $\mathcal{F}(N) = Sym(T)$ and its witnesses are runs of T. Partition certification computes $Sym(T)$ by Theorem 2d. (b) By Definition 7, a confirmed finding comes with a run of T that exhibits the key (for a robust key: misses the property under at most f failures, which is a robust or goal key of T by Definition 3). (c) Example 1. □

Theorem 4. (a) We give the families and show that every slice smaller than the claimed depth fails to confirm; a slice of the claimed size confirms. *S2*: a fork into $m \geq 2$ branches, branch i a chain of $\ell_i \geq 1$ agents whose first agent claims goal g_i, joined by a gate J with quorum $q < m$. Static findings are empty (every goal is claimed upstream, no requirements). In a singleton $\{v\}$ every entry carries $cone(v)$; J's singleton sees the union of its inputs, and no other node is a switch or fails, so $AL = \emptyset$ (Proposition 1). The run in which J uses q branches other than i misses g_i; in the slice $\{b_i, J\}$ of the last agent of branch i and the join, b_i's entry carries g_i at face value and J can choose the other entries, and the team run with the same choice at J lifts it. So the depth is 2. *S3*: as S2 with a fact and a consumer after the join. In all families, the ℓ intermediate nodes are agents with empty requirements and guarantees, each on its own oracle, so they create no symptom of their own, and any slice that contains neither of the two interacting nodes sees both claims at face value. *S4*: two agents r_1, r_2 on one oracle, both claiming a critical goal, separated by ℓ such agents, with $f = 1$. A slice that contains only one of r_1, r_2 can fail only that one; the other is outside and applies at face value, so the critical goal arrives. A slice containing both contains their hull, of size $\ell + 2$; failing the shared oracle removes both claims, and the team run with the same failure lifts it. *S6a*: an agent a providing x, ℓ hand-offs, a gate G requiring x and guaranteeing a critical goal. A slice without a sees x at face value; a slice without G has G outside, applying at face value. The hull of $\{a, G\}$ has $\ell + 2$ nodes and confirms the robust key. *S6b*: a gate y that fails to provide p, an agent a requiring p and guaranteeing z, ℓ hand-offs, a gate G requiring z and guaranteeing a goal. (assume, a, p) is static, but a slice without G has G applying at face value, and a slice without a sees z at face value; the hull of $\{a, G\}$ confirms the goal key. The test suite and the depth sweep (E4b) check these families for $\ell \leq 5$. (b) Let P be a view predicate and let the analysis flag a team iff some view satisfies P. If the analysis is sound, the unsafe team is flagged, so some view u of it satisfies P. By construction u is also a view of one of the safe teams, which is then flagged: a false alarm. The constructions (S4: reviewers on oracles (o_1, o_2) and (o_2, o_1) versus (o_1, o_1); S2: an all-join with split claims and an any-join whose branches claim both goals, versus an any-join with split claims; S6a: a gate provider, and a fallible provider with a backup gate, versus a fallible provider alone) are in the replication package and checked by E1. (c) Static persistence: in a run of H_σ that reaches v, its content is below its face-value content (Lemma 4 with $\emptyset \subseteq \sigma$), so the static requirement is still unmet (L4). Recovery of confirmed findings is Theorem 5c with $\{v\} \subseteq \sigma$. □

Theorem 5. (a) N confirms every symptom (Theorem 3a); confirming slices form a finite nonempty family, which has inclusion-minimal elements. (b) The fork-join example has two minimal slices. If $MMS(\kappa) = \{\sigma\}$, the intersection is σ. Conversely, a nonempty intersection of

convex sets is convex; if it confirms κ, it is a confirming subset of each minimal slice, so all minimal slices equal it. (c) Let ρ be the lifted run of T for $\kappa \in C(\sigma)$, ρ_σ its replay in H_σ (which exhibits κ), and $\rho_{\sigma'}$ its replay in $H_{\sigma'}$ (choices of ρ on σ', failures $F \cap orc(\sigma')$). Apply Lemma 4 twice. With $\sigma' \subseteq N$: the symptom node x is reached in ρ, so it is reached in $\rho_{\sigma'}$. With $\sigma \subseteq \sigma'$: ρ_σ is the restriction of $\rho_{\sigma'}$ (its failures are $(F \cap orc(\sigma')) \cap orc(\sigma) = F \cap orc(\sigma)$), so x's content in $\rho_{\sigma'}$ is below its content in ρ_σ, and the property is still missing (L4). Hence $\rho_{\sigma'}$ exhibits the same missing property at x, and ρ lifts it. For a robust key, the replay on σ has failures $F \cap orc(\sigma) \neq \emptyset$, so the replay on σ' has failures $F \cap orc(\sigma') \supseteq F \cap orc(\sigma)$, and the key is an (unsuppressed) robust candidate of σ'. In all cases ρ lifts κ for σ'. Upward closure and the depth characterization follow. (d) The team of Section 3: in $\sigma = \{s, x, y\}$ the output is outside and reached on one input, which gives a raw goal key; in $\sigma \cup \{\omega\}$ the output requires both inputs and no run of $H_{\sigma\cup\{\omega\}}$ reaches it without g. □

Proposition 6. Keys are triples of kind, node and property, defined independently of σ; merging is set union, and the localization keeps, per key, the list of confirming slices. The last claim is Definition 12 with Theorem 5c. □

Theorem 7. (a) Assume and goal keys: a run is a polynomial certificate, evaluated in linear time; Lemma 1 shows that runs suffice. Hardness is the CNF encoding in Section 3: for each assignment there is exactly one run (the switches), in which variable i contributes exactly one of t_i, f_i; clause gate j applies iff all its literals are false; so ω misses bad iff no clause is false, i.e. iff the assignment satisfies the formula. Robust keys: "some run with at most f failures misses φ" is in NP and "no fault-free run misses φ" is in coNP, so membership is in D^p. NP-hardness: after the clause chain, add a gate P that requires bad and guarantees the critical goal φ, followed by an agent A (on its own oracle) that also guarantees φ, with $f = 1$. Fault-free, A always provides φ, so the goal key is absent; failing A's oracle misses φ iff no clause is falsified, i.e. iff the formula is satisfiable. CoNP-hardness: the token construction of Section 3, where the robust key is present iff the formula is unsatisfiable. Safety: T is unsafe iff some fault-free run misses a goal or some run with at most f failures misses a critical goal, which is in NP, with hardness from the goal reduction. Reachability: a run is a certificate; for hardness, let one switch per variable choose a literal node, let one any-join per clause receive the literal nodes that satisfy it, and let an all-join R receive all clause nodes; R is reached iff the formula is satisfiable. E1 checks all four constructions against a SAT oracle. Depth: a certificate for $depth(\kappa) \leq k$ is a convex slice σ with $|\sigma| \leq k$ and a run ρ of T; checking that ρ exhibits κ and that its replay on σ exhibits κ takes polynomial time. Hardness: N confirms every symptom, so $depth(\kappa) \leq |N|$ iff $\kappa \in Sym(T)$, which is NP-hard for assume and goal keys. (b) Each internal run evaluates members once and propagates to outside descendants once, each in $O(|N| + |E|)$ word operations on property bit sets; filtering minimal contributor sets is quadratic in their number. Lifting searches runs of T with member switches fixed, which in the worst case is whole-team checking. (c) An antichain of w nodes has 2^w convex subsets; for fixed k, enumerating subsets of size $\leq k$ and testing $hull(U) = U$ with bit sets gives $O(|N|^k)$ tests. (d) Each strategy emits at most a constant number of slices per node (window, band, region) or per requirement (goal-directed), each a hull computed in $O(|N| + |E|)$. □

Theorem 8. (i) Oracle, kind, routing and policy of a enter H_σ only if $a \in \sigma$, since outside nodes are at face value. (ii) $req(a)$ and $decl(a)$ enter the applicability of a (only if $a \in \sigma$) and the keys at a; keys at a are dynamic findings of σ only if a is a member or below a member, i.e. if $\sigma \cap anc^*(a) \neq \emptyset$. (iii) A change of $gua(a)$ changes face-value outputs and run contents only in the bits of Δ. These bits affect applicability and keys only at nodes y with $req(y) \cap \Delta \neq \emptyset$ (for ω: goals), all in $desc(a)$, and at a through its output. If σ contains no ancestor-or-self of a node of Y, no member applies differently, no key at a node of Y is a dynamic finding of σ (it would lie below a member), and all other contents differ only in irrelevant bits. (iv) An edge change (u, v) changes the predecessors of v

and the successors of u; the content of v and of its descendants changes only in bits of u's face-value output, and ancestor relations change only for v's descendants and u's ancestors. Taking Y as stated and ancestors in both versions covers every member whose behavior or environment can change and every node whose keys can change. The conformance experiment compares incremental and full re-verification after random node and edge changes on every wild and injected team. □

B Team Specification Language

The replication package represents a team as a JSON document, loaded by `Team.from_json` in `slicing/model.py`. The document describes an architecture and its declared contracts; it does not contain prompts, executable tools, or measured model accuracies. The same representation supports chains, trees, parallel branches, switches, and joins, provided that the graph is acyclic and satisfies the well-formedness conditions of Definition 1. This appendix explains how to construct such a model and run the implemented analyses.

B.1 Fields and Defaults

The following schema summarizes the accepted fields. Bracketed default annotations are explanatory notation rather than JSON syntax; actual files use ordinary JSON objects and arrays.

Listing 1. Team specification schema. Optional fields carry their parser defaults.

```
team ::= {
  "name": string,                [default "team"]
  "input": [property],           [default []]
  "goals": [property],           [default []]
  "critical": [property],        [default []]
  "fault_budget": integer,       [default 0]
  "output_join": join,           [default "all"]
  "nodes": [node],
  "edges": [[source_id, target_id]]
}
node ::= {
  "id": string,
  "kind": "agent" | "gate",      [default "agent"]
  "req": [property],             [default []]
  "gua": [property],             [default []]
  "decl": [property],            [default []]
  "oracle": string,              [required for agents]
  "route": "fork" | "switch",    [default "fork"]
  "join": join                   [default "all"]
}
join ::= "all" | "any" | "quorum:q" | positive_integer
property ::= string
```

Properties and contracts. A property is an atomic name whose meaning is supplied by the modeler: for example, `code`, `reviewed`, or `tests_pass`. There is no implicit entailment between names. In particular, `spec@raw` and `spec@summary` are distinct properties; the suffix records a view but does not perform a conversion. A node's `req` lists the facts required to establish its `gua`. When reached, a node passes on its contributing input facts and adds its guarantees only if its requirements are met and, for an agent, its oracle has not failed. Otherwise, it forwards the contributing facts without adding guarantees. The local `decl` facts can discharge that node's requirements, but are

not forwarded. Thus a declaration that an agent has seen a specification does not itself supply that specification to downstream agents.

Agents, gates, and failure domains. Use `agent` for a fallible reasoning step and `gate` for a deterministic step under the model. Every agent needs an explicit `oracle` identifier when loading JSON. Agents with the same identifier share one failure domain: one failure can suppress all of their guarantees. Distinct identifiers encode distinct failure domains, not evidence of statistical independence. The nonnegative `fault_budget` bounds the number of failed oracle identifiers in a run. The `critical` array must be a subset of `goals`; these are the goals additionally checked under failures. A deterministic gate's guarantee remains conditional on its requirements, so declaring a test executor as a gate does not make an LLM-written test artifact reliable.

Edges, routing, and joins. Each edge transfers the source's output to the target. A node with `route: "fork"` delivers to every successor. A `switch` chooses exactly one successor; the checker explores these choices rather than evaluating a routing predicate. A switch with fewer than two successors is normalized to a fork. Routing and joining are independent attributes of a node. An `all` join needs every incoming edge to deliver and unions their contents. An `any` join needs one delivered edge; a quorum of q needs at least q delivered edges. The semantics explores admissible contributing subsets of delivered inputs, so an `any` join must not be interpreted as concatenating every available result. An integer quorum and its `quorum:q` spelling are parsed to the same policy. Choose $1 \leq q \leq d$ for a node of in-degree d.

Boundary nodes. If absent from `nodes`, `IN` and `OUT` are created as gates. The created `IN` guarantees `input`; the created `OUT` requires `goals` and uses `output_join`. Their incident edges are *not* inserted automatically: the edge list must connect them to the team. If these nodes are explicitly provided, their node fields take precedence; in particular, `OUT.req` must equal `goals`, and input facts belong in `IN.gua`. The optional name identifies the team in results and does not affect its semantics.

B.2 Constructing a Team

A model can be built from an orchestration design in five steps.

(1) *Choose the analysis boundary and property vocabulary.* State which facts are supplied at entry and which claims the final artifact must satisfy. Distinguish the existence of an artifact from its quality: `tests` and `tests_pass` express different claims.
(2) *Create one node per modeled step.* Assign unique identifiers, distinguish agent reasoning from deterministic gates, and state each step's requirements, guarantees, and any local declarations. A guarantee is a modeling assumption, not something inferred from a prompt.
(3) *Connect the actual hand-offs.* Add edges from `IN` through the steps to `OUT`. Set each branch's routing and every merge's join policy explicitly when the defaults do not match the orchestration. Every node must lie on a path from the input to the output. A retry loop must be unrolled to a finite acyclic model; shared mutable state is outside the language.
(4) *Choose failures and output obligations.* Give agents the same oracle identifier exactly when the model treats their failures as shared. Select the fault budget and critical goals. Increasing the budget changes the question being checked, without changing the contracts.
(5) *Validate and analyze.* Load the JSON, check structural errors, and check whether every node can be reached in some fault-free run. Structural validity alone does not establish liveness: mutually exclusive switch branches feeding an `all` join can leave that join unreachable. Then compare exact team symptoms with confirmed slice findings and inspect the returned witnesses.

Listing 2 gives a complete fork–join model. A writer provides code to a reviewer and a security tester. Both branches feed a release gate, which is configured to accept either branch. The model asks for both approvals, even though the gate can forward only one. All unspecified node fields take the defaults above, and the boundary nodes are created by the loader.

Listing 2. A complete JSON team with a release gate that accepts either approval. Save as team.json.

```
{
  "name": "release-on-one-approval",
  "input": ["spec"],
  "goals": ["reviewed", "sec_tested"],
  "critical": [],
  "fault_budget": 0,
  "nodes": [
    {"id": "Writer", "oracle": "writer_model",
     "req": ["spec"], "gua": ["code"]},
    {"id": "Reviewer", "oracle": "review_model",
     "req": ["code"], "gua": ["reviewed"]},
    {"id": "Tester", "oracle": "test_model",
     "req": ["code"], "gua": ["sec_tested"]},
    {"id": "Release", "kind": "gate", "join": "any"}
  ],
  "edges": [
    ["IN", "Writer"],
    ["Writer", "Reviewer"], ["Writer", "Tester"],
    ["Reviewer", "Release"], ["Tester", "Release"],
    ["Release", "OUT"]
  ]
}
```

B.3 Loading, Checking, and Revising the Model

Listing 3 uses the public Python functions in the replication package; run it from the repository root. Unknown edge endpoints and cycles are rejected during construction. The explicit validation call checks boundary connections, missing agent oracles, excessive quorums, and the consistency of goals. Identifiers must be unique before loading: the constructor stores nodes by identifier, so a repeated identifier would overwrite an earlier node rather than define a second component. The loader is not a general JSON-schema validator; callers should also use the documented kind and route values and nonnegative fault budgets.

The exact analysis reports the missing-goal keys `(goal, OUT, reviewed)` and `(goal, OUT, sec_tested)`. The confirmed two-component slices expose the relationship between the release policy and one of its input branches. Replacing the release policy by `all` makes both approvals contribute, and the repaired model has no symptom at fault budget zero. This is a statement about the declared model, not about the quality of reviews or tests in an execution.

The same construction supports other team shapes by changing the node and edge arrays rather than the checker. A chain uses one successor per step; parallel workers share a fork predecessor; a selector uses a switch; and an aggregator uses an all, any, or quorum join. To ask about robustness, add selected goals to `critical`, set a positive fault budget, and assign oracle identifiers consistently with the intended failure domains. Changing the release join alone does not guarantee safety under model failures. For larger teams, replace exhaustive enumeration by a chosen candidate family, for

Listing 3. Validate the team, compute exact symptoms, and confirm exhaustive slices of at most two components.

```
import json
from slicing.model import Team
from slicing import analysis, semantics, slices

with open("team.json", encoding="utf-8") as stream:
    team = Team.from_json(json.load(stream))
errors = team.validate()
if errors:
    raise ValueError(errors)
unreachable = set(team.nodes) - semantics.live_nodes(team)
if unreachable:
    raise ValueError(sorted(unreachable))

symptoms = semantics.symptoms(team)
family = slices.exhaustive(team, 2)
report = analysis.check_confirmed(team, family)
print("Exact symptoms:", sorted(symptoms))
for key, finding in sorted(report["findings"].items()):
    print(key, team.nodes_of(finding["top"]))

repaired = team.replace("Release", join="all")
assert not semantics.symptoms(repaired)
```

example `slices.window(team, 2)`, `slices.region(team)`, or `slices.goal_directed(team)`, while retaining `check_confirmed` when a reported finding must lift to a team run. A bounded candidate family can miss symptoms; run limits, lift-budget exhaustion, and time-outs must be read from the report's statistics rather than treated as a proof of safety. Exact whole-team checking also grows with the team's choice points.

References

[1] Martín Abadi and Leslie Lamport. 1993. Composing specifications. *ACM Transactions on Programming Languages and Systems* 15, 1 (1993), 73–132. doi:10.1145/151646.151649

[2] Andrea Arcuri and Lionel Briand. 2011. A practical guide for using statistical tests to assess randomized algorithms in software engineering. In *Proceedings of the 33rd International Conference on Software Engineering*. 1–10. doi:10.1145/1985793.1985795

[3] A. Avizienis. 1985. The N-Version Approach to Fault-Tolerant Software. *IEEE Transactions on Software Engineering* SE–11, 12 (1985), 1491–1501. doi:10.1109/tse.1985.231893

[4] Thomas Ball, Mayur Naik, and Sriram K. Rajamani. 2003. From symptom to cause: localizing errors in counterexample traces. In *Proceedings of the 30th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 97–105. doi:10.1145/604131.604140

[5] Albert Benveniste, Benoît Caillaud, Dejan Nickovic, Roberto Passerone, Jean-Baptiste Raclet, Philipp Reinkemeier, Alberto Sangiovanni-Vincentelli, Werner Damm, Thomas A. Henzinger, and Kim G. Larsen. 2018. *Contracts for System Design.* now Publishers Inc. doi:10.1561/9781680834031

[6] Marco Bozzano, Alessandro Cimatti, Cristian Mattarei, and Stefano Tonetta. 2014. Formal Safety Assessment via Contract-Based Design. In *Automated Technology for Verification and Analysis (ATVA 2014), Lecture Notes in Computer Science*. Vol. 8837. 81–97. doi:10.1007/978-3-319-11936-6_7

[7] Muffy Calder, Mario Kolberg, Evan H. Magill, and Stephan Reiff-Marganiec. 2003. Feature interaction: a critical review and considered forecast. *Computer Networks* 41, 1 (2003), 115–141. doi:10.1016/s1389-1286(02)00352-3

[8] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. 2025. Why Do

Multi-Agent LLM Systems Fail? arXiv:2503.13657. https://arxiv.org/abs/2503.13657

[9] Alessandro Cimatti and Stefano Tonetta. 2015. Contracts-refinement proof system for component-based embedded systems. *Science of Computer Programming* 97 (2015), 333–348. doi:10.1016/j.scico.2014.06.011

[10] Edmund Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. 2003. Counterexample-guided abstraction refinement for symbolic model checking. *J. ACM* 50, 5 (2003), 752–794. doi:10.1145/876638.876643

[11] Jamieson M. Cobleigh, Dimitra Giannakopoulou, and Corina S. Păsăreanu. 2003. Learning Assumptions for Compositional Verification. In *Lecture Notes in Computer Science*. 331–346. doi:10.1007/3-540-36577-x_24

[12] Darren Cofer, Andrew Gacek, Steven Miller, Michael W. Whalen, Brian LaValley, and Lui Sha. 2012. Compositional Verification of Architectural Models. In *NASA Formal Methods (NFM 2012), Lecture Notes in Computer Science*. Vol. 7226. 126–140. doi:10.1007/978-3-642-28891-3_13

[13] Stephen A. Cook. 1971. The complexity of theorem-proving procedures. In *Proceedings of the third annual ACM symposium on Theory of computing - STOC '71*. 151–158. doi:10.1145/800157.805047

[14] Luca de Alfaro and Thomas A. Henzinger. 2001. Interface automata. *ACM SIGSOFT Software Engineering Notes* 26, 5 (2001), 109–120. doi:10.1145/503271.503226

[15] Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An Efficient SMT Solver. In *Lecture Notes in Computer Science*. 337–340. doi:10.1007/978-3-540-78800-3_24

[16] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. arXiv:2406.13352. https://arxiv.org/abs/2406.13352

[17] Yihong Dong, Xue Jiang, Zhi Jin, and Ge Li. 2024. Self-Collaboration Code Generation via ChatGPT. *ACM Transactions on Software Engineering and Methodology* 33, 7 (2024), 1–38. doi:10.1145/3672459

[18] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. 2023. Improving Factuality and Reasoning in Language Models through Multiagent Debate. arXiv:2305.14325. https://arxiv.org/abs/2305.14325

[19] Bradley Efron and R.J. Tibshirani. 1994. *An Introduction to the Bootstrap*. Chapman and Hall/CRC. doi:10.1201/9780429246593

[20] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M. Zhang. 2023. Large Language Models for Software Engineering: Survey and Open Problems. In *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*. 31–53. doi:10.1109/icse-fose59343.2023.00008

[21] Jeanne Ferrante, Karl J. Ottenstein, and Joe D. Warren. 1987. The program dependence graph and its use in optimization. *ACM Transactions on Programming Languages and Systems* 9, 3 (1987), 319–349. doi:10.1145/24039.24041

[22] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*. 79–90. doi:10.1145/3605764.3623985

[23] Alex Groce, Sagar Chaki, Daniel Kroening, and Ofer Strichman. 2006. Error explanation with distance metrics. *International Journal on Software Tools for Technology Transfer* 8, 3 (2006), 229–247. doi:10.1007/s10009-005-0202-0

[24] Junda He, Christoph Treude, and David Lo. 2025. LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–30. doi:10.1145/3712003

[25] Sture Holm. 1979. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics* 6, 2 (1979), 65–70. https://www.jstor.org/stable/4615733

[26] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2023. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. arXiv:2308.00352. https://arxiv.org/abs/2308.00352

[27] S. Horwitz, T. Reps, and D. Binkley. 1988. Interprocedural slicing using dependence graphs. *ACM SIGPLAN Notices* 23, 7 (1988), 35–46. doi:10.1145/960116.53994

[28] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large Language Models for Software Engineering: A Systematic Literature Review. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 1–79. doi:10.1145/3695988

[29] Dong Huang, Jie M. Zhang, Michael Luck, Qingwen Bu, Yuhao Qing, and Heming Cui. 2023. AgentCoder: Multi-Agent-based Code Generation with Iterative Testing and Optimisation. arXiv:2312.13010. https://arxiv.org/abs/2312.13010

[30] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabsa. 2023. Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations. arXiv:2312.06674. https://arxiv.org/abs/2312.06674

[31] Md. Ashraful Islam, Mohammed Eunus Ali, and Md Rizwan Parvez. 2024. MapCoder: Multi-Agent Code Generation for Competitive Problem Solving. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. 4912–4944. doi:10.18653/v1/2024.acl-long.269

[32] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2023. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? arXiv:2310.06770. https://arxiv.org/abs/2310.06770

[33] Richard Johnson, David Pearson, and Keshav Pingali. 1994. The Program Structure Tree: Computing Control Regions in Linear Time. In *Proceedings of the ACM SIGPLAN 1994 Conference on Programming Language Design and Implementation (PLDI)*. 171–185. doi:10.1145/178243.178258

[34] C. B. Jones. 1983. Tentative steps toward a development method for interfering programs. *ACM Transactions on Programming Languages and Systems* 5, 4 (1983), 596–619. doi:10.1145/69575.69577

[35] James A. Jones and Mary Jean Harrold. 2005. Empirical evaluation of the tarantula automatic fault-localization technique. In *Proceedings of the 20th IEEE/ACM International Conference on Automated Software Engineering*. 273–282. doi:10.1145/1101908.1101949

[36] John B. Kam and Jeffrey D. Ullman. 1977. Monotone data flow analysis frameworks. *Acta Informatica* 7, 3 (1977), 305–317. doi:10.1007/bf00290339

[37] Gary A. Kildall. 1973. A unified approach to global program optimization. In *Proceedings of the 1st annual ACM SIGACT-SIGPLAN symposium on Principles of programming languages - POPL '73*. 194–206. doi:10.1145/512927.512945

[38] John C. Knight and Nancy G. Leveson. 1986. An experimental evaluation of the assumption of independence in multiversion programming. *IEEE Transactions on Software Engineering* SE–12, 1 (1986), 96–109. doi:10.1109/tse.1986.6312924

[39] Thomas Lengauer and Robert Endre Tarjan. 1979. A fast algorithm for finding dominators in a flowgraph. *ACM Transactions on Programming Languages and Systems* 1, 1 (1979), 121–141. doi:10.1145/357062.357071

[40] Nancy G. Leveson. 2012. *Engineering a Safer World: Systems Thinking Applied to Safety*. MIT Press. doi:10.7551/mitpress/8179.001.0001

[41] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society. arXiv:2303.17760. https://arxiv.org/abs/2303.17760

[42] B. Littlewood and D.R. Miller. 1989. Conceptual modeling of coincident failures in multiversion software. *IEEE Transactions on Software Engineering* 15, 12 (1989), 1596–1614. doi:10.1109/32.58771

[43] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. 2023. AgentBench: Evaluating LLMs as Agents. arXiv:2308.03688. https://arxiv.org/abs/2308.03688

[44] Alessio Lomuscio, Hongyang Qu, and Franco Raimondi. 2015. MCMAS: an open-source model checker for the verification of multi-agent systems. *International Journal on Software Tools for Technology Transfer* 19, 1 (2015), 9–30. doi:10.1007/s10009-015-0378-x

[45] K. L. McMillan. 1999. Circular Compositional Reasoning about Liveness. In *Correct Hardware Design and Verification Methods (CHARME 1999), Lecture Notes in Computer Science*. Vol. 1703. 342–346. doi:10.1007/3-540-48153-2_30

[46] B. Meyer. 1992. Applying 'design by contract'. *Computer* 25, 10 (1992), 40–51. doi:10.1109/2.161279

[47] J. Misra and K.M. Chandy. 1981. Proofs of Networks of Processes. *IEEE Transactions on Software Engineering* SE–7, 4 (1981), 417–426. doi:10.1109/tse.1981.230844

[48] Kedar S. Namjoshi and Richard J. Trefler. 2010. On the completeness of compositional reasoning methods. *ACM Transactions on Computational Logic* 11, 3 (2010), 1–22. doi:10.1145/1740582.1740584

[49] Laura Pozzi, Kubilay Atasu, and Paolo Ienne. 2006. Exact and approximate algorithms for the extension of embedded processor instruction sets. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 25, 7 (2006), 1209–1229. doi:10.1109/TCAD.2005.855950

[50] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ChatDev: Communicative Agents for Software Development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. 15174–15186. doi:10.18653/v1/2024.acl-long.810

[51] Traian Rebedea, Razvan Dinu, Makesh Narsimhan Sreedhar, Christopher Parisien, and Jonathan Cohen. 2023. NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. 431–445. doi:10.18653/v1/2023.emnlp-demo.40

[52] Raymond Reiter. 1987. A theory of diagnosis from first principles. *Artificial Intelligence* 32, 1 (1987), 57–95. doi:10.1016/0004-3702(87)90062-2

[53] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. 2023. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. arXiv:2309.15817. https://arxiv.org/abs/2309.15817

[54] Enno Ruijters and Mariëlle Stoelinga. 2015. Fault tree analysis: A survey of the state-of-the-art in modeling, analysis and tools. *Computer Science Review* 15–16 (2015), 29–62. doi:10.1016/j.cosrev.2015.03.001

[55] Wei Tao, Yucheng Zhou, Yanlin Wang, Wenqiang Zhang, Hongyu Zhang, and Yu Cheng. 2024. MAGIS: LLM-Based Multi-Agent Framework for GitHub Issue Resolution. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*. 51963–51993. doi:10.52202/079017-1647

[56] W. M. P. van der Aalst. 1998. The Application of Petri Nets to Workflow Management. *Journal of Circuits, Systems and Computers* 08, 01 (1998), 21–66. doi:10.1142/s0218126698000043

[57] András Vargha and Harold D. Delaney. 2000. A Critique and Improvement of the "CL" Common Language Effect Size Statistics of McGraw and Wong. *Journal of Educational and Behavioral Statistics* 25, 2 (2000), 101. doi:10.2307/1165329

[58] Haoyu Wang, Christopher M. Poskitt, and Jun Sun. 2025. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. arXiv:2503.18666. https://arxiv.org/abs/2503.18666

[59] Junlin Wang, Jue Wang, Ben Athiwaratkun, Ce Zhang, and James Zou. 2024. Mixture-of-Agents Enhances Large Language Model Capabilities. arXiv:2406.04692. https://arxiv.org/abs/2406.04692

[60] Mark Weiser. 1984. Program Slicing. *IEEE Transactions on Software Engineering* SE–10, 4 (1984), 352–357. doi:10.1109/tse.1984.5010248

[61] Frank Wilcoxon. 1945. Individual Comparisons by Ranking Methods. *Biometrics Bulletin* 1, 6 (1945), 80. doi:10.2307/3001968

[62] W. Eric Wong, Ruizhi Gao, Yihao Li, Rui Abreu, and Franz Wotawa. 2016. A Survey on Software Fault Localization. *IEEE Transactions on Software Engineering* 42, 8 (2016), 707–740. doi:10.1109/TSE.2016.2521368

[63] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155. https://arxiv.org/abs/2308.08155

[64] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2025. Demystifying LLM-Based Software Engineering Agents. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 801–824. doi:10.1145/3715754

[65] Asaf Yehudai, Lilach Eden, Alan Li, Guy Uziel, Yilun Zhao, Roy Bar-Haim, Arman Cohan, and Michal Shmueli-Scheuer. 2025. Survey on Evaluation of LLM-based Agents. arXiv:2503.16416. https://arxiv.org/abs/2503.16416

[66] A. Zeller and R. Hildebrandt. 2002. Simplifying and isolating failure-inducing input. *IEEE Transactions on Software Engineering* 28, 2 (2002), 183–200. doi:10.1109/32.988498

[67] Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Wu. 2025. Which Agent Causes Task Failures and When? On Automated Failure Attribution of LLM Multi-Agent Systems. arXiv:2505.00212. https://arxiv.org/abs/2505.00212